QR-Shield: A Malicious QR Code Detection Tool



Authors

Faheem Ahmed	F22BINCE1M04106
Mehmood Ul Hassan	F22BINCE1M04050
Isha Fatima	F22BINCE1M04033

Supervisor

Dr. Iftikhar Rasheed

A final year project report submitted in partial fulfillment of the requirements for the degree of

B.S. Cyber Security and Digital Forensics

Department of Information and Communication Engineering

Faculty of Engineering and Technology

The Islamia University of Bahawalpur, Pakistan

Declaration of Originality

I declare that the research work titled *"QR-Shield: A Malicious QR Code Detection Tool"* is my own work and has not been submitted previously for any degree or examination. All sources of information used in this thesis have been properly acknowledged.

Faheem Ahmed

Mehmood Ul Hassan

Isha Fatima

Certificate

This is to certify that the thesis entitled *"QRShield: A Hybrid Machine Learning and Deep Learning Approach for Secure QR Code Detection"* has been prepared by **Faheem Ahmed (F22BINCE1M04106)**, **Mehmood Ul Hassan (F22BINCE1M04050)**, and **Isha Fatima (F22BINCE1M04033)** under the supervision of **Dr. Iftikhar Rasheed** in partial fulfillment of the requirements for the degree of **B.S. Cyber Security & Digital Forensics**, Department of Information and Communication Engineering, Faculty of Engineering and Technology, The Islamia University of Bahawalpur, Pakistan.

Dr. Iftikhar Rasheed (Supervisor)

External Examiner Name (External Examiner)

Dr. Muhammad Ali Qureshi (Chairman)

Abstract

QR codes are widely used in digital payments, marketing, authentication, and information sharing, but they are also vulnerable to malicious redirects, phishing attacks, and visual tampering. This thesis presents *QRShield*, a hybrid security framework that combines machine learning-based URL analysis with deep learning-based QR image tampering detection to improve the detection of malicious QR codes.

The proposed system first extracts the embedded URL from a scanned QR code and analyzes it using an XGBoost classifier trained on 27 security-related features derived from lexical properties, WHOIS data, DNS records, and HTTP behavior. In parallel, the QR image is passed through a ResNet-18-based tampering detector to identify distortions, overlays, and other suspicious visual modifications. The outputs of both models are combined through a decision-level fusion strategy to produce the final classification as benign, suspicious, or malicious.

For evaluation, two team-generated datasets were used: the Malicious URLs Features Dataset and the Binary QR Codes Dataset. The URL classification branch achieved strong performance, while the QR tampering detector also showed high accuracy in distinguishing clean and distorted QR images. The results demonstrate that combining payload analysis with visual inspection provides a more robust and practical approach to QR code security.

Keywords: QR Code Security, Malicious URL Detection, Machine Learning, Deep Learning, Visual Tampering Detection, Cyber Security

Acknowledgments

We are profoundly grateful to Allah Almighty for granting us the strength, wisdom, and perseverance to complete this research work.

We sincerely thank our supervisor, **Dr. Iftikhar Rasheed**, for his invaluable guidance, continuous support, and constructive feedback throughout this project. We also extend our gratitude for his valuable suggestions and encouragement.

We further acknowledge the **Department of Information and Communication Engineering**, **Faculty of Engineering and Technology**, and **The Islamia University of Bahawalpur, Pakistan**, for providing the academic environment and resources necessary for this work.

Finally, we thank our parents, friends, and group members for their support, cooperation, and encouragement throughout this journey.

Contents

Declaration of Originality i

Certificate ii

Abstract iii

Acknowledgments iv

List of Figures ix

List of Tables x

1 Introduction 1

1.1 Background and Motivation 1

1.2 Need for Automated Secure QR Code Detection 2

1.3 Problem Statement 3

1.4 Research Objectives 5

1.5 Research Questions 6

1.6 Scope of the Study 7

1.7 Limitations 7

1.8 Significance of the Study 8

2 Literature Review 10

2.1 Introduction to QR Code Security 10

2.2 Malicious URL Detection Techniques 11
2.3 QR Code-Based Phishing (Quishing) Attacks 12
2.4 Visual Tampering and Image-Based QR Analysis 13
2.5 Feature Engineering and Dataset Construction 14
2.6 Model Architectures and Training Strategies 15
2.7 Multimodal and Fusion Approaches 17
2.8 Deployment, Evaluation, and Adversarial Considerations 18
2.9 Research Gaps and Summary 20

3 Methodology **23**

3.1 Introduction 23
3.2 System Overview 24
3.3 QR Detection and Decoding 25
3.4 Dataset Usage in QRShield 26
3.5 URL Feature Engineering 27
3.6 XGBoost URL Classification Model 30
3.7 Training Procedure for the Machine Learning Branch 31
3.8 ResNet-18 Tampering Detection Model 31
3.9 Training Procedure for the Deep Learning Branch 32
3.10 Fusion Strategy 33
3.11 Evaluation Metrics 38
3.12 Summary 40

4 System Implementation and Deployment **42**

4.1 Introduction 42
4.2 Overall Implementation Strategy 43
4.3 Web Application Implementation 43
4.4 Mobile Application Implementation 44

4.5 Backend Architecture 45
4.6 QR Detection and Decoding Module 46
4.7 URL Analysis Module 48
4.8 QR Tampering Detection Module 49
4.9 SHAP-Based Explainability Module 50
4.10 Fusion and Final Decision 52
4.11 Deployment of Web and Mobile Clients 53
4.12 Implementation Benefits 56
4.13 Summary 57

5 Results and Discussion **58**

5.1 Introduction 58
5.2 URL Classification Results 59
5.3 Visual Tampering Detection Results 60
5.4 Discussion of the URL Model Results 61
5.5 Discussion of the QR Image Model Results 62
5.6 Overall System Interpretation 63
5.7 Explainability and SHAP Discussion 64
5.8 Limitations Observed in the Results 65
5.9 Summary 66

6 Conclusion and Future Work **68**

6.1 Research Summary 68
6.2 Contributions of the Study 69
6.3 Achievement of Research Objectives 70
6.4 Limitations 71
6.5 Future Work and Recommendations 73
6.6 Concluding Remarks 75

References **76**

List of Figures

1.1 Classification of Modern Threats Targeting QR Code Systems 2

3.1 Architecture of the proposed QRShield System 25

3.2 Fusion Strategy of the proposed QRShield System 37

4.1 Qr Code Detection and Decoding Pipeline 47

4.2 QrShield Pro Website Interface 1 54

4.3 QrShield Pro Website Interface 2 54

4.4 QrShield Pro Website Interface 3 55

4.5 QrShield Pro Website Interface 4 55

List of Tables

2.1 Comparison of QR Code Security Detection Approaches 20

3.1 Dataset Description 27

3.2 Extracted URL Security Features from URL [?] 28

4.1 SHAP Feature Interpretation Example 51

5.1 URL Classification Performance Comparison 59

5.2 QR Image Classification Performance Comparison 60

5.3 Fusion-Based Performance Comparison 64

6.1 Research Objectives Achievement Summary 71

Chapter 1

Introduction

1.1 Background and Motivation

Quick Response, or QR, codes, are popular for quick and convenient information transfer. User-friendliness is one of the several reasons for their popularity. For a user to access the information contained within them, they simply need a camera-enabled device (or a QR code scanner). Almost instant access to information stored digitally makes QR codes a great physical-digital object connector.

This useful feature of QR codes makes them a great tool for many industrial systems; however, they also pose challenges for security systems. Plain text links reveal end destinations, but the end of a QR code is unknown until the QR code is fully processed. Because of this, a valid form of verification to assess if the inscribed content is webpage is a malicious one or a safe one is impossible. This makes QR codes potentially very dangerous tools for phishing attacks, and malware delivery attacks. QR codes are designed as a simple black-and-white square which prompts many users to trust them. This lack of suspicion makes QR codes an even more effective tool for cyber attacks.

Intentional distortion of QR codes raises another level of concern. An attacker can obscure part of a target QR code, overpaint part of the original QR code, and/or create a distorted image that sends users to a different URL. Restaurants, bus stops, payment and information boards are public places where users scan QR codes. Users tend to scan codes without verifying the source; therefore, QR codes can be used for phishing attacks, fraud, and social engineering.

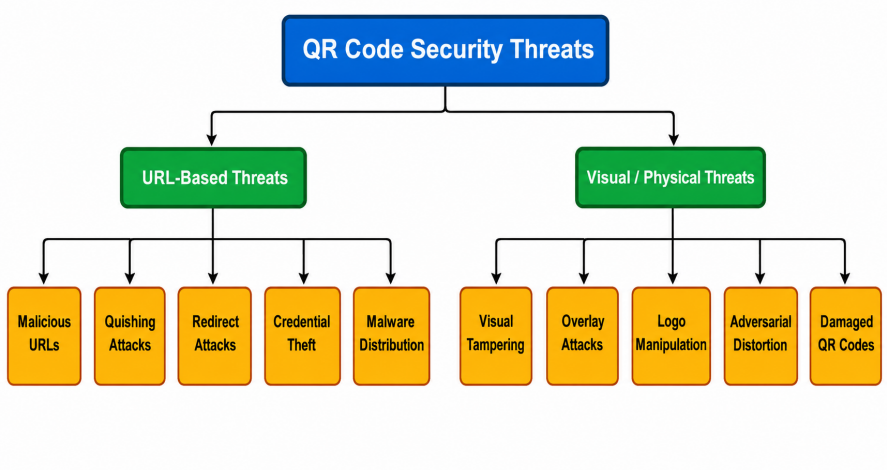


Figure 1.1: Classification of Modern Threats Targeting QR Code Systems

This study revolves around the challenges posed to the attacker and the concern to be defended for the victim. Using existing systems cannot be defended, as they tend to address only portions of the problem. For example, some systems check the URL after decoding the QR code, and others check the QR code for Tampering and distortion. However, in the case of QR codes, what matters is both a malicious URL and a distorted QR code. It is necessary to develop such a complex system that enables the analysis of both aspects of the QR code, i.e., the content and the image.

To tackle this problem QRShield has been developed. It is an adaptive security framework that relies on machine learning and deep learning techniques to enhance protection for QR codes. The system subjects the embedded URL to an XGBoost classifier that is trained on engineered security features, while it also employs a ResNet-18 model to assess if the QR image has been subjected to some form of visual tampering. QRShield intends to offer a more reliable and workable solution to the contemporary QR-based threats by combining both types of evidence.

1.2 Need for Automated Secure QR Code Detection

Manual inspection based risk assessment for QR codes is an unreliable detection method. Data encoded in QR codes is not visible to the human eye. It may show a URL after the QR code is scanned, but determining the safety of the URL is still a challenge. Attackers employ deceptive techniques to maliciously present QR codes. URL shortening, subdo-

main tricks, slight domain tomfoolery, and redirecting chains are some methods used by attackers. These methods are designed to evade casual inspections and be undetectable to the human user.

Under the time constraints of real-life environments, QR codes are usually scanned as is. Restaurant patrons may scan a code on the dining table to view the menu, users may scan a payment code, or someone may scan a shown code because they think it is safe. Such instances mean that the time related to a decision may be a greater cost than a risk, especially as the user may not have the knowledge to understand code manipulation. Hence, it is imperative to have an automated system.

Automation also provides consistency. It can process QR codes at high speed using a set of rules and framework. Automation will be incredibly valuable as the system can be embedded in a mobile app, web app, a security control, or forensic technology and be used in everyday scenarios and environments where great automated system optimization is needed.

More support for automation comes from the fact that QR code attacks may not be contained to their URLs. One QR code may appear just like any other, but may contain a web link to a malicious website. Even in a case where the web link is not disguised to appear malicious, the malicious nature of the QR code may be embedded within a changed image. It is clear that a detection system cannot be limited to a single mechanism. All factors like the URL, the structure, and possible tampering of a QR code image need to be analyzed to devise an effective security solution.

This is the exact scenario that QRShield aims to address. Whether analyzing the content of a QR code or the QR code image, the system is designed to perform both of these tasks in one single application. It will also analyze the QR code and alert a user to the danger of a potentially malicious QR code. QRShield is tailored for effective real-world use in both web and mobile environments. This applicability will ensure that the same detection principles will be maintained on both platforms.

1.3 Problem Statement

The more people leverage QR codes, the more security issues start to arise. Today QR codes are being incorporated into payments, logins, tickets, information about a product,

advertisements, and even public services. Unfortunately, this level of utility gives way for attackers to conceal misbehavior within a format that appears to be engaged with a level of trust. It allows for QR codes to be easily modified to point towards nefarious sites, phishing logins, and even sites that host malicious files for distribution.

Conventional methods to ensure security are too limited to address these attacks. Methods that rely on the code scanning public to be informed about the issues with code scanning are severely flawed as once a code is scanned in a public space, it is very likely the destination will be checked only by the first code scanner. Methods that rely on a blacklist or filter to assess the safety of shortened / obfuscated links or new attacks against the system are doomed to the same fate. Lastly, the QR image may be inspected for distortion or alteration, and may allow for the detection of a tampered code. Protection is insufficient as providing a mechanism to assess the safety of the question is outside of it.

The primary concern that this work addresses is the lack of automated systems that ensure a hybrid QR code security system. In order to ensure hybrid QR code security a system needs to assess safety of the URL encoded within a QR code, and determine if the QR code was compromised. These two concerns are interrelated, but neither can be neglected, with the focus on only one leaving the system vulnerable to an attack along the other concern.

Another concern is practicality. A viable security solution would need to run in the real world, as opposed to just theory. This would mean the solution would need to handle QR codes, run clear outputs for users, and support integration into mobile and web applications. An explainable solution increases the system's usability by supporting users to better understand the justification for the QR codes flagged as suspicious or malicious.

The challenge, thus, is to build a hybrid QR code security system that analyzes QR images and embedded URLs, effectively identifies malicious and/or altered QR codes, and meets a timeliness and usability constraint. QRShield is a viable solution to this challenge by integrating machine learning, deep learning, and explainable artificial intelligence into one system.

1.4 Research Objectives

The purpose of this study is to create a hybrid QR code security system, QRShield, that uses a combination of ML and DL methods to recognize harmful URLs and detect altered QR codes. With the constant rise in the use of QR codes and the exploitation of QR codes by a rising number of attackers, this research becomes necessary. Since it is not revealed where a QR code directs until it is scanned, it can be used to mask a Phishing page, harmful content, or a redirected page that is malicious and not to be seen. Hence, the suggested system aims to analyze the embedded URL and the visual state of the QR code. A two-stream pipeline is demonstrated in this research, where the URL branch is handled by XGBoost and the image branch is handled by ResNet-18.

The goals for this study are first to evaluate and analyze the risks that QR codes pose in security and assess potential attacks. This will consider how QR codes may lead to URLs which are malicious, and look at QR based phishing, visual tampering, redirections, and URL shorteners and other similar harmful techniques. This study also aims to create a URL classification framework using XGBoost which will help detect malicious and suspicious links which are QR codes. Alongside, a Phishing QR code 'alteration testing' model is created using ResNet-18 to help determine if the QR code in question is altered, manipulated, or even overlaid QR code. Once developed, these models will then be combined together to a single hybrid system which will allow a final answer to be automated. In the methodology section, URL tampering and ResNet-18 detection will also serve as the foundation of QRShield for this research.

The other important goal is to measure the effectiveness of the designed framework using the datasets produced by the team for the project. In this case, standard performance metrics will be used. These will be accuracy, precision, recall, and F1 Score, as well as a confusion matrix. If possible, ROC-AUC will also be used. The collection of results is to demonstrate that the models work individually, and also show that the hybrid provides improved QR security detection. The goal of the study is to show that a combined approach is more robust than single-modality/either-or approaches using URL security or image security.

1.5 Research Questions

Several research questions primarily guide this thesis. The first is the effective detection of malicious URLs within QR Codes using machine learning. This question is significant since attackers obfuscate, shorten, redirect, or use crafted domain names that disguise the harmful URLs. To answer this question, a learning model has to differentiate between malign URLs and benign URLs using both lexical and infrastructural URL features.

The second question is the detection of tampered or altered QR Code images using deep learning. This question emphasizes the visual aspect of the problem, as the QR Code may be altered by overlay, cropping or distortion among other forms of manipulation. Since the visual representation of a QR Code may not be consistent with the actual content which is embedded, a model is needed to facilitate the reliable detection of such alterations.

The third question is the assessment of the security of QR Codes using a hybrid model as compared to a single model. This is a focal question since QRShield is designed to use both URL-based and image-based evidence. The assumption is that one of the two branches will capture a threat that the other will not, thereby enhancing the overall reliability of the system. The paper constructs this theory by describing QRShield as a complementary multimodal framework in which both branches present different but valuable security signals.

The fourth question analyzes classification accuracy and reliability performance for the provided datasets using QRShield. It focuses on the practical performance of the system and the generalization of the chosen models on the available data for the project. The fifth question revolves around the potential of such a system for advancing digital security and forensic examination. QR code attacks go beyond the safety of the end user issues and also affect forensic and investigative casework, particularly when criminal codes are disseminated in public or organized environments.

1.6 Scope of the Study

This research investigates the security risks of QR codes and focuses on malicious URLs and visual tampering. There are two main analysis dimensions. The first is the threat analysis of URLs. Here, the embedded link in a QR code is assessed using lexical features, WHOIS, DNS, HTTP, and more, to determine if the URL is malicious or not. The second analysis focuses on QR Code image tampering. In this dimension, a deep learning model is utilized to detect Edge, Overlay, Occlusion, and other types of suspicious modifications to the QR code. The paper provides clear divisions on this research, one for URL security and one for QR image security.

In terms of the scope of the study, the implementation is limited to the datasets that the project team collected and developed for the QRShield system. Given that the study is primarily of a research nature, there is no commercial intent. Therefore, the aim of the study is to design, develop, and test the hybrid detection framework, and not to develop a scanning enterprise-grade platform. The study also focuses on a back-end type analysis, where the QR image is sent to a server for further analysis, thereby shifting the focus of the study to the system's design, and the model's and framework's performance and detection capabilities, rather than the optimization of a product at scale from the app store or a hardware-level edge deployment. The authors also describe that their back-end architecture is intended to allow for parallel analyses and support rapid processing and low latency.

In general terms, the study does not address every possible QR related security concern. It does not encompass all cybercrimes, all methods of cybercrime carrying malware, or all sophisticated adversarial mobile attacks. The study focuses on the specific and most pertinent problem of QR codes that lead to an embedded malicious URL, or QR code image that has been altered. By limiting the scope of the problem, the study will be able to provide a practical and feasible solution considering the constraints of time and resource of a final year project.

1.7 Limitations

While QRShield can improve QR code security, there are some drawbacks. The first is the balance of the training and testing data. If the data do not adequately represent

real world scenarios, the model can struggle to generalize to examples that were not in the training data. As the attackers may constantly be changing their techniques, for example, by using a different style of a malicious URL, or by a different style of tampering with images, robustness is required. As the paper also mentions, increasing the number and variety of the data available will improve future system versions.

The second limitation involves the dependencies for some URL features on external information resources. While WHOIS and DNS added features are helpful, in the real world, these lookups can be delayed, be rate-limited, be only partially available, or be inconsistent with URL domain data. This means that feature extraction may not be fully accomplished. When such dependencies are lacking, the system needs to use the remaining features more heavily, which can affect the robustness of the system. The paper also mentions the dependence on external lookup as a URL security system generic practical limitation.

A significant limitation of the current approach is that it focuses on QR code URL and QR image tampering detection. Although still important, these are not the only threats that can be used with QR codes. Dynamic redirecting within a code, and other forms of more sophisticated obfuscated QR based code, can still present threats that a static system is unable to address. Furthermore, a system that relies on backend processing will always introduce latency and, in case of low-bandwidth environments, or offline access, will severely limit usability. For these aforementioned reasons, the current solution is largely a quality framework for QR threats detection, and is not a comprehensive solution for all possible threats.

1.8 Significance of the Study

This study is important as it confronts a real world issue of increasing cybersecurity risk. QR codes are ubiquitous and are increasingly becoming part of our societies as a result of their use in finance, communication, education, and even transport, advertising, and public services. Unfortunately, where there is a greater use, there is a greater potential for misuse as well. QR codes can be used to direct users to phishing websites, initiate malicious downloads, or present a deceptive login. Most users won't suspect anything until it is too late. Therefore, a system that can analyze a QR code and detect threats

prior to users interacting with it demonstrates significant security advantage.

In this regard, QRShield is the first of its kind system that integrates two different, but complementary, methods of machine learning: one for detecting threat URLs integrated with another for detecting QR code tampering. This kind of design is crucial for reducing the disadvantages of systems that primarily depend on one method. For example, if the URL is suspicious, the code can still be flagged even if the image looks fine. Conversely, if the image is suspicious, the code can be flagged even if the URL is fine. This dual processing indeed improves the reliability of the system and its overall security. The paper positions QRShield exactly in this regard. It is a hybrid framework that combines payload analysis with visual integrity analysis.

This study shows how hybrid intelligence can be used for security issues with an academic viewpoint, while also integrating machine learning, deep learning, and explainability into one framework. Practically, this system supports safer scanning in real-world environments and even works through web and mobile interfaces. Because of this, it can be used by everyday users as well as security-conscious organizations. Since you also have SHAP explainability in your system, it has even more value as it is not a black box and can actually explain why a QR code is classified as risky. For this reason, this work is also relevant to digital forensics and trustworthy AI, in addition to cybersecurity.

Chapter 2

Literature Review

2.1 Introduction to QR Code Security

QR codes link the physical and digital worlds like no other communication tool. They can instantly provide information, direct users to websites, or connect to online services. QR codes can even facilitate payment. This makes QR codes faster and easier for the user than most other communication mechanisms. They just point their camera or a scanning app at the QR code. However, this speedy and easy access to information can be exploited for malicious purposes. [1, 2, 3]

QR codes can conceal the destination of the communication, creating a security risk. Plain url hyperlinks can be scanned to determine the destination. However, this is not the case for QR codes. They can link to a phising website, a malware download website, or to a site that is completely different than what the QR code claims it will. Additionally, QR codes can be altered in ways that a user might not even notice. Malicious users can place stickers over the QR code, or even download a QR code online, and place it in its place. QR codes can even be altered and a malicious user can cut portions of the QR code, and QR code patterns can be unexpectedly distorted. Even users are told not to trust the QR code, it looks like nothing is out of the ordinary. [2, 4, 5]

It is not possible to handle QR security by analyzing it from just one angle. A URL checker can overlook some forms of visual attack. Analyzing the QR image alone can overlook a well-formed QR code that contains a malicious payload. That is why most of the literature considers QR security a multimodal problem. This motivates the expan-

sion of research into several areas, such as malicious URL detection, quishing, image-based attack detection, feature extraction, model creation, combining different modes of attacks, and even the implementation of solutions. Based on the multimodal attack research, the QRShield framework combines URL-based machine learning and an image-based deep learning approach to detect malicious payloads and visual QR code manipulation. [3, 5, 6]

2.2 Malicious URL Detection Techniques

Malicious URLs obtained by scanning QR codes present unique challenges to security researchers. During such analyses, researchers treat URLs as primary indicators of a QR code's safety. Researchers examine the URL's structure and other information related to the domain and server. The main feature categories include lexical, WHOIS, DNS, and HTTP features. Lexical features analyze the visible shape of the URLs. They analyze the URL's length, number of dots/ hyphens/ digits/ and special characters, presence of suspicious keywords (e.g., login or secure), and use of IP addresses in the URL instead of domains. WHOIS features examine domain registration information, and DNS features examine domain resolution and the presence of MX records and the number of IP addresses associated with the domain. HTTP features analyze the URL's responses, redirects, and the presence of an SSL certificate. By themselves, the features outlined in the categories provide useful and crucial information about the safety of a URL. However, in analyses of the URLs, the categories of features are most useful when all are used in a complementary manner. [7, 8, 9]

The literature documents extensive use of machine learning models in this area, including but not limited to Logistic Regression, Support Vector Machines, Random Forests, Gradient Boosting, and XGBoost. These models are trained on tabular URL data extracted from QR codes or collected from phishing data. Among these models, the tree-based ensemble methods exhibit the greatest popularity, as they can deal with data of different types and are often the best performers on structured security data. This is because malicious URL determination is not a one hint problem. Rather, it is a multi-hint problem, as there are a number of combined suspicious structural patterns, infra behavioral signs, and server anomalous activities. The tree-based ensemble methods are particularly good at learning these patterns. The analysis in your file also mentions that

Random Forests and XGBoost are among the top performers when used with engineered lexical, WHOIS, DNS, and HTTP features, in detecting malicious URLs. [6, 10, 9]

Nevertheless, the literature documents extensive and important short-comings of URL only based models. URL based filtering is often trivial to circumvent, e.g., by using obfuscation, shortening and redirection services, and homograph domain attacks. Often a URL may be structurally benign, but lead to a malicious site after several hops. In such cases a filter based only on the URL will be ineffective, especially in the case of manipulated QR Code images. This is the main reason that QRShield is a combined URL and image analysis framework and not a URL only filtering framework. [4, 2, 5]

2.3 QR Code-Based Phishing (Quishing) Attacks

Phishing that uses QR codes, called 'quishing,' is an emerging threat. Users tend to scan QR codes without considering the code to determine the destination. In a phishing example, the user at the very least sees the intended malicious link. With a QR code, the user simply can't inspect anything because the code contains an image, the payload, and the destination encrypted. Because there is less information available, this added layer of obfuscation makes the QR code attack even more effective. Some methods attackers use to increase the trust of the user in the QR code is placing the malicious code over a QR code that has information, using printed menus and posters, and more. In most cases, the user is in a hurry therefore quishing attacks tend to be successfully executed in the real world. [2, 8, 4]

The quishing models available in the literature have to depend on small data samples that lack analysis features. Because of the lack of quality and unavailability of data, in many cases the data have to be analyzed to show the captured conditions, print quality, and so on. With regard to quality and reproducibility, these are the major limitations to the models. With a QR code, an attacker may use a malicious endpoint, a short link, and multiple link resolutions to unveil the intended endpoint. Because of these characteristics, QR-specific phishing is more difficult to detect compared to a typical phishing attack, causing greater threat exposure to a user. [11, 2, 3]

This is why the literature encourages QR-specific security solutions in addition to standard URL filtering. A QR-focused security system should be able to understand and mit-

igate the unique behaviors associated with QR exploits, such as URL shorteners, URL redirections, and hidden payloads. It must also address real-world scenarios wherein users scan QR codes in public with very little oversight. This is the reason the thesis structure focuses on payload and visual tampering analysis. Therefore, QRShield is designed to counteract an attack in the way it is most likely to be presented to the user: a QR code that is/or may be digitally or physically altered, and may lead to a dangerous destination. [5, 12, 3]

2.4 Visual Tampering and Image-Based QR Analysis

The second primary method of research is aimed at uncovering visual alterations to QR codes. In this case, the QR code is an analytical image problem, instead of solely a URL classification dilemma. The concept is that a QR image can uncover alterations without even assessing the URL. Examples can include alterations such as noise, scratches, overlays, spacial or edge warping, or even light or printing artifacts if they represent some form of occlusion or incomplete coverage of the QR Code. These give evidence that the QR Code may be a diversion, changing the user's target URL, amongst other things. Additionally, they may show that the code no longer serves the purpose of its initial creator. Therefore, the QR Code may be a unique form of image security. [13, 14, 3]

Due to the necessity for critical application of QR codes, many of the mentioned techniques and methods derived from the basic principles of image processing apply to QR Code recognition. The primary methods in the aforementioned articles rely on convolution networks, as they are optimally equipped to recognize and differentiate between visual and pixel-based alterations or patterns that are made during the capture and framing process. Examples of capture variability may include noise from unintentional occlusion, captured blur and light inconsistencies, or edge occlusion. These methods are key for a model to differentiate between the variability that is inherent and the variability that is the result of design or malicious intent. In the real world, QR Codes are often photographed under less than ideal circumstances. A QR code may be captured from a surface outside of a viewer's normal line of sight, captured with uneven light, captured from an oblique angle or even distorted or captured in a printed form. All of these effects of the augmentation can be helpful in lessening the effects of real world obstacles or variation to a model that learns from a QR code. [13, 14, 15]

The reviewed work states that depending on the goal, different model families are employed. Whenever the system needs to work on-device or with limited resources, lightweight models mobile net variants can be used. If server side analysis is done and we can afford better and stronger feature extraction, residual networks e.g., ResNet-18 or ResNet-34 are frequently employed. Although all of these models are useful in detecting tampering, they all unfortunately have the same limitation, and that is detecting if the decoded URL is indeed malicious. A QR image may look distorted and suspicious, and actually still be pointing to a legit URL, whereas another QR image may look clean but still be pointing to a phishing page. This is where image detection is not adequate for achieving complete QR safety, hence a hybrid system such as QRShield is warranted. [13, 15, 3]

2.5 Feature Engineering and Dataset Construction

The efficiency of any QR security detection system is closely associated with the caliber of features and the quality of the datasets that are available for the training and testing processes. In the context of QR security-related tasks, feature engineering assumes a greater significance because most adversarial actions do not necessarily manifest in clear and obvious ways with the QR image. In URL-based analysis, researchers extract a combination of lexical, WHOIS, DNS, and HTTP-based features, among others, because malicious URLs often display patterns associated with multi-layered complex malicious activities. Lexical features evaluate the visible construction of the URL, for example, the URL length, number of dots, number of hyphens, number of digits, special characters, and presence of suspicious words like login, secure, verify, or update. WHOIS features provide info on the domain registration, especially domain age and ownership, and are useful because newly registered or hidden domains are often used for malicious acts. DNS features evaluate whether a domain resolves, whether it has MX records, and how many IP addresses it is associated with. HTTP-based features evaluate a URL's behavior to determine a count of redirects, presence of SSL and status codes. These categories provide a richer and more robust representation of a URL than any one category alone. [9, 8, 7]

The literature indicates that designing a feature set with approximately 20 to 40 features proves efficient for malicious URL detection for data obtained from QR codes or related

to phishing, as depicted in multiple case studies. Malicious URLs demonstrate a combination of weak infrastructure signals, suspicious response behavior, and an abnormal response that cannot be detected with the help of a single feature. A collection of several features makes it possible to bring out the hidden characteristics and patterns that differentiate malicious links from normal links. With these reasons in mind, QRShield uses a 27-feature representation in the URL branch since it provides a realistic balance for use in an implementation and avoids unnecessary complexity. [9, 11, 3]

There is a similar need for well-constructed datasets for QR image analysis. For QR code visual tampering detection, it is necessary to avoid relying solely on clean, synthetic (artificially constructed) examples. Real-world images may be captured in suboptimal lighting, be out of focus due to motion blur, or may exhibit changes in print quality due to degradation, compression artifacts, or be distorted. If the dataset lacks such examples, the model is likely to fail when deployed for real-world use, even if it performs well in a controlled setting. Researchers mitigate this risk through synthetic perturbations such as blurring, adding occlusion, altering brightness, adding noise from compression, and warping from changes in perspective. These augmentations help the model learn to differentiate between capture variability and tampering. [13, 14, 15]

The publications also highlight careful labeling and reliability of datasets. For URL datasets, labels may be generated from a combination of manual reviews, blacklists, sandbox crawling, and other external verification methods. This helps decrease labeling noise and increases the training data's reliability. For QR image datasets, labels must be created to efficiently separate clean and tampered images to ensure the image model does not learn the wrong visual patterns. The work noted the importance of caching WHOIS and DNS lookups. The repeated external queries can be slow and inconsistent. It is clear that proper feature engineering and meticulous creation of datasets are important for the successful implementation of the entire QR security framework. [9, 8, 3]

2.6 Model Architectures and Training Strategies

When it comes to tabular data containing URL features, the majority of studies show a preference for tree-based ensemble models. This is due to their ability to deal with structured data that can. be of mixed feature types, and due to the nonlinear relationships

that often exist within the data. Random Forest, Gradient Boosting, and particularly XGBoost, are commonly applied models for URL and phishing detection, owing to their performance, interpretability, and practicality. XGBoost stands out for its ability to deal with missing values, model complex decision boundaries, and to work with mixed features without the need for extensive preprocessing, i.e., feature scaling. This modeling ability is relevant to QR security because URL representations may combine a mixture of formats, including binary, count-based, and numeric. The paper also cites that Logistic Regression and Support Vector Machines are commonly used to set a performance baseline. However, when the data has complex feature interactions, tree-based ensemble models tend to outperform these. [6, 10, 9]

XGBoost is a popular choice for malicious URL detection in research literature. This is due to the nature of the problem domain, where maliciousness is rarely a single indicator and is most often a combination of several indicators. For example, a URL can seem short and harmless, but be a malicious URL combining a suspicious redirection, recent domain registration, and absence of DNS or MX records. Ensemble tree methods are able to learn these critical and complex combinations. Because of this, they are a good fit for QR-derived URL classification due to the presence of many noisy signals. This is the rationale on which QRShield is built, and classifies URLs after extracting 27 security features using the XGBoost framework. [10, 7, 8]

Image-based QR tampering detection often relies on transfer learning due to limited size QR image datasets, and this is often the approach of choice in literature. Training a deep model on these small datasets from scratch is not a viable option, therefore leveraging a model that has been trained for a different purpose and then fine-tuning the model for QR tampering detection is a common approach. Some of the models used for this purpose are ResNet, DenseNet, and MobileNet. ResNet models (and their derivatives) are frequently more effective than other model architectures and allow highly complex and deep representations of the data to be learned. MobileNet is often the model of choice when a smaller model must be used for deep learning, and ResNet 18 and 34 are often used in cases where model resource constraints on the server are not present. [14, 13, 15]

Training strategies have the same significance in image-based QR analysis. Given that QR codes are exposed to camera artifacts and variations in the environment, training

usually incorporates augmentation for the sake of robustness. The reviewed literature mentions random rotation, perspective warp, Gaussian blur, JPEG compression, brightness and contrast jitter, and random occlusion as viable augmentations. These allow the model to learn the types of distortions that are present in real-world scenarios as opposed to overfitting to artificially created images that are clean. The framework of the paper, in this case, is using ResNet-18 as the image branch and training the model on clean QR codes as opposed to distorted/tampered QR codes. The literature supports the choice since ResNet-18 is a strong option in the accuracy-complexity tradeoff. [14, 13, 3]

2.7 Multimodal and Fusion Approaches

Because URL-based and image-based methods capture different pieces of evidence, research on QR security has shifted toward hybrid systems. This shift is evident in the literature that corroborates that protective measures in QR security cannot be satisfied with one method alone. A URL-only detector indicates a harmful destination, but does not indicate if the QR has been manipulated. An image-only system indicates a harmful destination, but does not verify if the QR code contains a harmful destination. A hybrid system is a more appropriate solution, as it combines the benefits of both systems and minimizes the drawbacks of each system. URL and image evidence together yield a more comprehensive solution. This is the approach adopted by QRShield, where URL manipulation and image tampering detection are fused to provide the final classification. [5, 6, 15]

The literature identifies three main fusion strategies. The first is rule-based fusion, where a fixed rule determines the final outcome. For example, a QR code may be examined as malicious if the URL classifier identifies a harmful link, or if the Image classifier identifies a malicious tampering. This approach is simple and easy to understand, making it appealing in security applications where transparency is critical. The next strategy is score-based fusion, where the final decision is made using weighted averages of the confidence scores of both classifiers. This allows both classifiers to participate in the decision proportionately. The last of the three strategies is meta-classifier fusion, where the outputs of both branches are combined to feed a model that learns to fuse these outputs. This approach is more flexible in that it can capture more complex interactions

between image and URL evidence. [5, 3, 2]

The studies reviewed indicate that multimodal fusion enhances recall in quishing detection potentially because attackers may be able to fool one branch but not the other. An example may include a QR code that is visually normal while a malicious URL is hidden, and/or a QR code that is visually suspicious but is still resolving to a malicious destination. Fusing both signals may prevent false negatives and allow the system to provide better protection. This is one of the foremost reasons hybrid systems are the preferred option in QR security research. The studies indicate that fusion methods should be robust if one source of evidence is missing. For example, in a QR code, WHOIS information may be unavailable, or the code may not be completely decoded. Fusion methods should ensure that the solutions implemented remain sufficiently robust in cases of missing evidence as is common in real world systems. [15, 3, 5]

The studies maintained that good fusion design must ensure usability and reliability. A perfect integrated fusion design must achieve a balance between sensitivity and precision. QRShield implements this via a decision-level fusion that integrates both the URL model and the image model. This makes the QRShield highly suitable for protecting against the various attacks implemented in real QR phishing. [5, 3]

2.8 Deployment, Evaluation, and Adversarial Considerations

Another essential topic when considering QR code security is practical implementation. A detection model can only be considered effective if it is viable in the context in which users scan QR codes. For this reason, many QR code security solutions use a backend-based architecture in which the client device (usually a mobile device or web interface) captures a QR code image and sends it to a server. The server then performs QR code localization, decoding, feature extraction, model inference, and final classification. This architecture has several advantages: the server can use more complex and powerful models, and the client application remains light. It also allows more complex mathematical models (e.g. machine learning or deep learning) that may not be suitable to run on a mobile device. [5, 3, 16]

There are important considerations with backend deployment. The first is concerning

latency. When a QR code first is transmitted to a backend server for analysis and then run through a machine learning model, the total response time will depend on more than just model inference. Time for the packet to travel through the network and time for the backend server to finish processing also add to the total time. This is especially crucial for a QR code scanning service, as users generally expect the result to be returned instantly. There are also concerns with privacy, as the scanned QR code and the potentially sensitive URL that was encoded in the QR code may be sent to a remote system for processing. Because of this, it has been suggested that not only the accuracy of classification be considered when evaluating a QR security system. End-to-end latency and throughput, as well as the memory footprint, and consistency across classes, should also be considered. This is because a system that has been validated to be accurate would not be good in a practical sense if it is resource intensive or overly slow. [3, 9, 5]

The importance of per-class confusion matrix analysis is highlighted in the literature. In some security-related applications, the positive predictive value can be misleading. A model, for instance, can render positive predictions for benign examples while failing to perform any detection of malicious examples. Following the QR security example, erroneously classifying a benign QR code as malicious is a much lesser concern than ultimately failing to detect a malicious QR code. For this reason, error analysis is important. Error analysis helps determine whether a model is operating under a closed world assumption, is exhibiting class bias, and where additional improvements are warranted. Concerning QRShield, where the classification of QR Codes falls within benign, suspicious, and malicious with varying degrees of concern, this type of analysis is important. [9, 11, 3]

The literature pegs adversarial robustness as the second largest concern. The QR security system needs to accommodate clean examples as well as adversarial cases. Adversarial cases may include QR codes where the link is a shortened URL and adversarial intent is cloaked or a QR code where the link directs users to an otherwise benign domain. In the context of QR codes, adversarial acts may include altering the QR code to add a disruptive pattern or paint over the code to obscure it. These examples are designed to exploit the weaknesses of the system. [2, 12, 3]

Due to these vulnerabilities, a realistic QR security system can only be developed if

tested under practical and hostile circumstances. Testing models against clean data sets would fail to reflect a model's capability against real-world threats, as clear data is the opposite of what attackers present. Therefore, the existing research supports testing systems against obfuscated URLs, distorted QR images, redirect chains, and capture noise in order to model robustness more accurately. This is really important for hybrid systems such as QRShield, as the model should remain effective even if one of the modalities is degraded or partially missing. In this context, deployment and adversarial evaluation should be placed as the primary priorities to demonstrate that the system is suitable for real-world security. [3, 2, 14]

Table 2.1: Comparison of QR Code Security Detection Approaches

Study	**Approach**	**Features Used**	**Detection Type**	**Limitations**
Detecting Quishing Attacks using ML	Machine Learning based URL classification	Lexical URL features	Malicious URL detection	Does not analyze QR image tampering
Efficient Malicious QR Code Detection System	Deep Learning based QR analysis	Image features from QR patterns	Visual tampering detection	Cannot detect malicious URLs embedded in QR codes
QSecR Secure QR Code Scanner	Rule-based QR code scanning	URL pattern rules	Phishing URL detection	Limited adaptability to new attacks
AI Enhanced Security Framework for QR Codes	Hybrid ML approach	URL + network features	Malicious link detection	No visual tampering detection
QRShield (Proposed Model)	Hybrid ML + DL framework	Lexical, WHOIS, DNS, HTTP + QR image features	Malicious URL + QR tampering detection	Processing Outside the System

2.9 Research Gaps and Summary

Based on the literature review, both URL-based detection and image-based tampering detection methods are helpful, though both have critical limitations when used in isolation. URL-only systems are adept at finding suspicious links, but cannot identify

whether or not the QR image has been modified. On the other hand, image-based systems can be used to identify visible alterations, but cannot determine if the decoded URL is in fact malicious. Ultimately, an exclusive reliance on either URL-based or image-based detection systems creates an inherent gap in defense, as QR attacks can be executed either via a malicious URL, the image, or both. This is why the reviewed studies assert that a combined solution is the only option for a more comprehensive and robust security solution. [3, 5, 6]

A significant gap in the literature is the limited availability of large, well-balanced QR datasets. A large portion of the studies are based on small, heterogeneous, or proprietary datasets, making it difficult to have an acceptable basis for comparison of mechanisms employed. Furthermore, QR security datasets vary in terms of how they capture images, label URLs, and simulate tampering, which makes it difficult to develop a reliable model that generalizes well in different real-world situations. According to the literature, some datasets are also lacking in diversity, which means that even when a model performs well on a given data set, it is likely to fail in generalizing to a even mildly different set of conditions and attack modalities. [3, 11, 15]

Another gap is in the design of the fusion strategies employed. Many of the hybrid systems, of which there are plenty, do not provide a clear description of how the URL and image outputs are combined. Some employ rule-based logic, some utilize score averaging, and some implement meta-classifiers. Nevertheless, these approaches do not handle missing or noisy inputs necessarily. For example, the system may have to make a decision when WHOIS data is missing, the URL is not fully extended, or the QR image is beyond decoding. Hence, the literature argues that design of the fusion be carried out such that the system would be functionally stable in the presence of an incomplete signal. [3, 5]

Another gap pertains to the reporting of constraints on deployment. Many of the papers report on the constructs of accuracy, precision, and recall. They do not provide a detailed description of latency, privacy, throughput, and memory management. This is troubling because a QR security system is of value once it is viable in the field as part of a mobile application, a web scanner, or a forensics tool. Otherwise, it may not be a security solution of any use, but remains a laboratory prototype. This is the reason why the architecture used on the back end of the system, the time taken to respond, and how

resources are consumed, are essential to the research problem. [3, 9, 5]

QRShield is developing a hybrid solution that integrates the capabilities of the URL branch and the Image branch alongside the decision-level fusion strategy. The URL branch captures the lexical structure and the infrastructure cues (WHOIS, DNS, and HTTP) and the Image branch captures the visual integrity of the QR Code. The integration of both the branches, through the decision-level fusion strategy, is a constructive move towards more tangible and hybrid solutions to this problem. The framework is well supported because it integrates both evidence of the payload and the image, which is more in line with attacks on QR Codes rather than single-modality solutions with limited scope. [3, 2, 5]

The reviewed literature demonstrates the need for a hybrid QR security framework. Due to the composite nature of the threats posed by QR codes, URL-only and image-only frameworks are insufficient. The reviewed studies advocate for a URL feature engineering, image tampering evidence, and rational fusion. The cited works propose the integration of these elements. QRShield hybridizes engineering, machine learning, and deep learning to create a more advanced detection system for malicious and altered QR codes. [3, 5, 2]

Chapter 3

Methodology

3.1 Introduction

This chapter contains information regarding the construction and the methodology of the proposed design for the QRShield framework. This system has been constructed to combat malicious QR code attacks using a two-part complementary analytical system. One arm of the system conducts threat analysis based on URLs, while the other conducts analysis to detect whether the QR code image has been tampered with. The cornerstone of this design argues that the security of a QR code can, and should, be assessed using multiple sources of evidence. The QR code may have a destination that is malicious, despite the code appearing to be calm and normal, and may be tampered with, even if the decoded URL appears to not be suspicious in any way. Using both these scenarios, QRShield has been constructed to use a combination of machine learning, as well as deep learning, in a single robust security framework.

The methodology employs a back-end oriented design to ensure that the actual detection process is both flexible and efficient. The design of the system ensures that once a QR code image is captured on the client side, the framework conducts the following processes in order without requiring the user to perform any other action. The first step is the localization of the QR code within the provided image. Once this is achieved, the system decodes the embedded QR code. If the decoded payload contains a link, the system extracts the URL and then performs a security analysis of the URL. This, along with the QR code image, is sent for further processing to the two branches of the framework.

Prior to this, the QR code image was classified by the URL payload using XGBoost within the URL branch, while within the image branch, a form of deep learning was employed using ResNet-18, to classify the QR code based image. The framework then integrates the outputs of both the branches to arrive at a final decision. The design goes a long way in providing reliability to aid the detection of both quishing and QR code image tampering attacks.

3.2 System Overview

QRShield uses a two-stream design for its backend. The first step is when a user scans a QR code and uploads the image through the app. The first step of the backend locates and decodes the QR code through computer vision. If the code is decoded, the backend provides the embedded URL to the URL analysis, and the backend crops and digitizes the QR code for the image analysis. The URL analysis uses XGBoost to produce a score related to the URL, and the image analysis uses ResNet-18 to determine a score for QR code tampering. Once the two scores are combined, the result determines the overall threshold, which is classified as benign, suspicious, or malicious. The paper claims this is a parallel asynchronous design and aims to optimize the total time of the process.

This system design incurs positive results due to its modular component design for QR codes. The first component preprocesses and decodes the QR code, URL analysis classifies and evaluates URL security, and image analysis determines the structural integrity of the QR code. The simultaneous analysis of the QR code and of the visual integrity of the QR code can be completed through QRShield and makes the design more flexible and applicable for real use conditions.

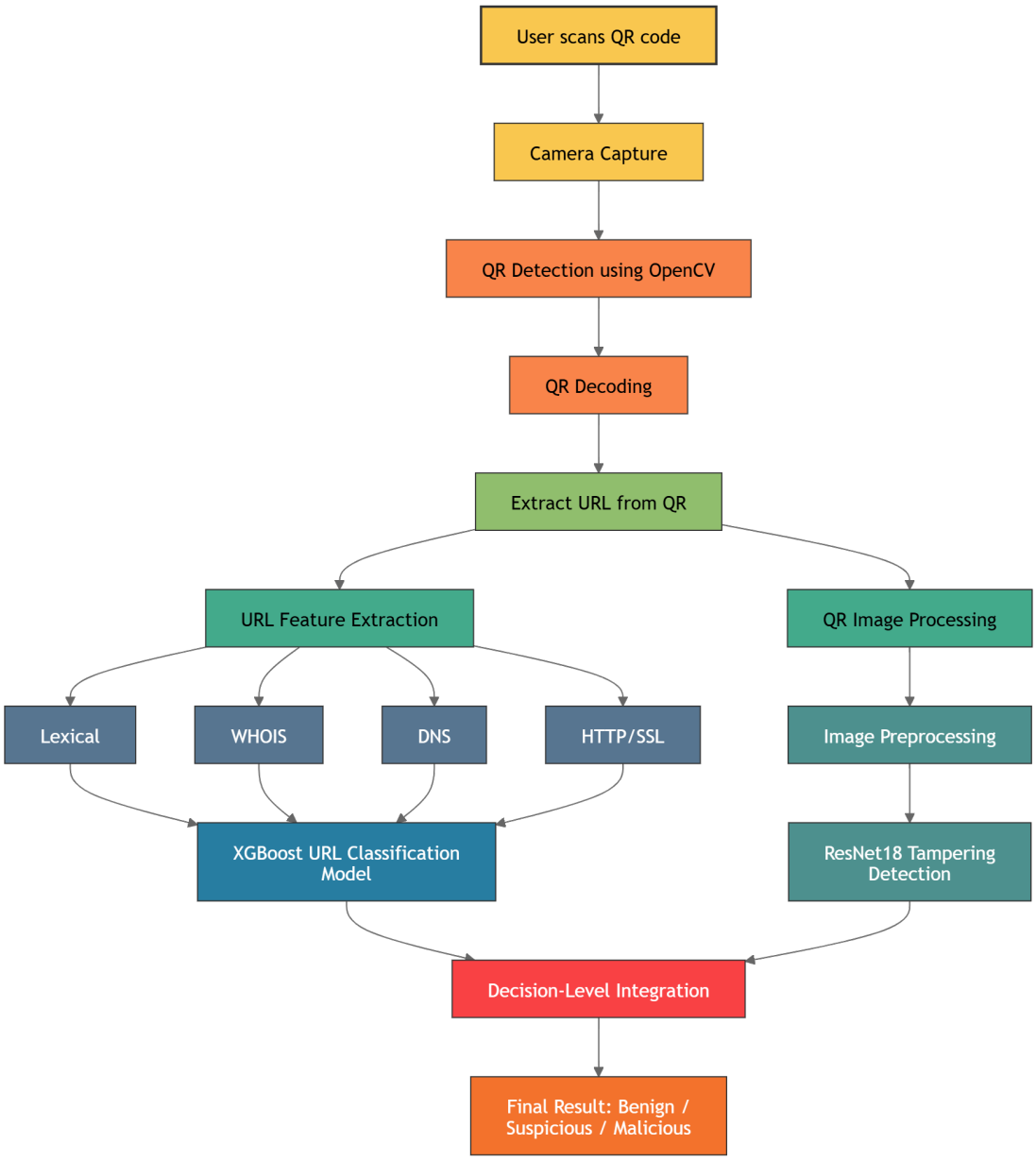


Figure 3.1: Architecture of the proposed QRShield System

3.3 QR Detection and Decoding

The first step in the process is to detect and decode the QR code. To do so, the system employs a number of image processing techniques which include converting the image to grayscale, adaptive thresholding, contour detection, and determining the location of the finder pattern. After this, the Open CV library is employed to draw the finder pattern and to localize the QR code. QR codes, like other codes and images, may be captured in

imperfect conditions, so this step is important. Blurred, skewed, and poorly lit images may partially cover the QR code and make them unsuitable for decoding.

Once the QR code image is isolated, the image is processed for optimal decoding, which may include standard image processing techniques like contrast enhancement and denoising with a bilateral filter. Decoding may still fail and in these cases the process is repeated with other image enhancement techniques like histogram equalization, image sharpening, and morphological closing in a series of retries to counteract the effects of less than optimal image capture. QR codes are thus decoded in a very flexible and resilient manner.

When a QR code has a shortened or redirected URL, our system views the shortened link as potentially dangerous. This is due to the fact that URL shorteners can hide a malicious target that can result in 'quishing'. Since we cannot adjust the shortened link for now, we give users a chance to manually intervene within our analysis and paste the link where needed. After this step, our analysis takes some of the available data that consists of WHIO, DNS, and HTTP lexical features, and decides the link's risk. This method is a great inclusion for our system, as we can treat all shortened URLs with caution due to their threats revolving around redirection, phishing and malicious obfuscation. This precaution includes measurement techniques.

3.4 Dataset Usage in QRShield

The proposed framework employs two distinct datasets. The first dataset is the Malicious URLs Features Dataset, which contains 1,000 labeled URLs represented by their extracted security features. This first dataset is used to train a machine learning model to classify malicious URLs. The second dataset is the Binary QR Codes Dataset, which contains both clean and distorted QR images, and is used to train a deep learning model to detect tampering. A dual-dataset design is incorporated so that QRShield is able to analyze both the content of the URL and the visual design of the QR code.

Table 3.1: Dataset Description

Dataset Name	**Purpose**	**Number of Samples**	**Classes**	**Description**
Team Eagle URL Dataset	URL Classification	1000	Benign, Malicious	Contains extracted lexical, WHOIS, DNS, and HTTP-based URL features
Team Eagle QR Code Dataset	QR Tampering Detection	Add Actual Count	Clean, Distorted	Contains QR images with normal and tampered visual patterns

Both datasets perform two distinct but interrelated tasks. The URL dataset is used after a QR code is decoded; at this point, the embedded link is passed through the feature engineering module and is then transformed to a structured form for the classifier. The image dataset is used simultaneously, where the QR code patch is resized and is then passed through the ResNet-18 model to perform tampering. This design enables both branches to specialize in a distinct section of the attack surface and simultaneously contribute to the fused decision. To elaborate, one dataset trains the system to evaluate the destination of the code, and the other trains the system to evaluate the destination of the code.

This dataset strategy is methodologically beneficial for corresponding with an actual threat model for QR attacks. A QR threat could emanate from malicious payload behavior, and/or physical and/or digital tampering. QRShield uses one dataset for URL risk and one for image integrity and, before merging them in the fusion stage, models both sides of the problem independently.

3.5 URL Feature Engineering

The URL analysis uses a set of 27 features, incorporating lexical, WHOIS, DNS, and HTTP-based signals, to create a structured representation of a URL. The goal with this feature design is to capture both the visible structure and the underlying infrastructural behavior of a URL. This is crucial because most malicious URL patterns are suspicious not only in the textual content of the link, but also in the domain registration and resolution patterns, and how the domain is hosted. The features listed in the paper include

checking$_ip_address, abnormal_url, count_dot, count_at, shortening_service, url_length, hostname_length, susp$

Lexical features are obtained from the URL string and describe its structure. They incorporate the visible format of the URL, including its length, hostname length, and the number of dots, digits, hyphens, special characters, and if suspicious keywords are present. These types of features are important since phishing URLs can be evident through abnormal URL formats. A common tactic used for crafting phishing URLs is creating an unusually long subdomain and/or including verbiage such as 'login,' 'verify/validate' (or other variations) in an effort to convey a sense of legit/secure service. These types of lexical features can classify URLs as 'abnormal' as a first step, even prior to any external features.

Table 3.2: Extracted URL Security Features from URL [**?**]

Feature Name	**Description**
checking ip address	Indicates whether the URL uses an IP address instead of a domain name.
abnormal url	Checks if the domain name appears in the URL structure, identifying abnormal URLs.
count dot	Total number of dots (.) present in the URL.
count at	Number of '@' symbols in the URL which may indicate redirection tricks.
find dir	Counts the number of directory separators ('/') in the URL path.
no of embed	Number of embedded '//' occurrences after the protocol in the URL.
shortening service	Indicates whether the URL uses a known URL shortening service.
count per	Counts the number of percent (%) encoding characters in the URL.
count ques	Number of question marks (?) indicating query parameters.
count dash	Number of hyphens (-) in the URL.
count equal	Counts the number of equal signs (=) in query parameters.
url length	Total length of the entire URL string.

Feature Name	**Description**
hostname length	Length of the domain hostname in the URL.
suspicious words	Detects presence of phishing-related keywords such as login, verify, or secure.
digit count	Number of numeric digits appearing in the URL.
count special chars	Total number of special characters present in the URL.
fd length	Length of the first directory in the URL path.
tld length	Length of the top-level domain (TLD).
uses https	Indicates whether the URL uses HTTPS protocol.
domain age days	Age of the domain in days obtained from WHOIS records.
whois available	Indicates whether WHOIS information is available for the domain.
dns resolves	Checks whether the domain successfully resolves via DNS lookup.
num ip addresses	Number of IP addresses returned for the domain in DNS resolution.
has mx record	Indicates whether the domain has a Mail Exchange (MX) record.
ssl valid	Indicates whether the URL uses a valid SSL connection.
redirect count	Number of redirections encountered when accessing the URL.
http status code	HTTP response status code returned by the web server.

WHOIS-based features provide information about how a domain was registered. One of these features is domain age. The newer the domain, the more likely it is connected to phishing. The available WHOIS information is helpful because, in many cases, registration data could be hidden to conceal the true owner. DNS-based features make it possible to look at how a domain can be resolved, is connected to an IP and also MX records, and how stable the domain's supporting infrastructure is. HTTP-based features analyze how many times a domain redirects, if a valid SSL is present, and how the domain responds to requests. The features whether grouped together, or looked at individually, provide the best information to the classifier.

The paper also draws a correlation heat map of the different URL features. The main

observation is the low (or absence of) correlation of the different features, which means that these features provide information that is complementary rather than redundant. This is a great attribute as it, in layman's terms, means that the model has many ways to provide a signal, and relies on the signal in a different way. In security classification, this diversity is even more important since that means that it's unlikely that an attacker was able to change all parts of a URL at the same time.

3.6 XGBoost URL Classification Model

The URL Classification Sub-Module uses the XGBoost Classifier, as it demonstrates superior performance with structured/tabular data. The XGBoost Classifier is provided with a 27-dimensional feature vector obtained from feature extraction, and it returns a probability score which quantifies the likelihood of a URL being malicious. This is vital for the QRShield pipeline, as XGBoost captures and converts potentially malicious security features of a URL into a format that a prediction can be later integrated with other outputs, such as image analysis. The URL Classification Module has been validated based on the traditional classification metrics, SPF score, as well as the ROC-AUC score, which are valid metrics for security classification problems.

The detection of malicious URLs is traditionally a classification problem that is non-linear, and XGBoost is the best-performing classification algorithm for this type of problem. A URL can be viewed as malicious for many different reasons that can be classified as non-linear. For example, these may include an abnormal structure, a short domain lifespan, weak DNS signals, an uncommon redirect, and other behaviors. Non-linear interactions among features and tabular data are well handled by boosting models. Also, feature scaling is less critical when tabular data are used which makes the models easier to deploy. Among the other classification algorithms compared in the paper, XGBoost, taking into consideration the other algorithms Random Forest, Logistic Regression, and SVM, exhibited the best URL classification, and this is why it has been selected for the final framework.

Another reason to pick XGBoost is that it can be adjusted to account for class imbalance, which is frequent in security datasets. The paper says class imbalance can be addressed with either class weights or oversampling, and using stratified splits or 5-fold

cross-validation can help evaluate model performance more effectively. It is crucial for security classification not only to be accurate, but to have the classification model be sensitive to the malicious cases more than the benign cases. In a real-world scenario, a classification model that predicts the majority class would be harmful, so XGBoost, with its adaptability, would be a great choice to pick for QRShield URL branch.

3.7 Training Procedure for the Machine Learning Branch

A URL model is developed using a modified stratified split of the Malicious URLs Features Dataset with 70, 15, and 15 percent divisions for training, validation, and testing, respectively. Since class distributions of malicious and benign URLs are often imbalanced, this approach aims to fairly evaluate the model and keep the distributions consistent.

This model overcomes URL dataset challenges, such as missing WHOIS and DNS values, by normalizing, log-normalizing, or using sentinel values. URL security features are heterogeneous and can be counts or binary or even depend on external lookups. Early stopping improves model validation performance on new examples and maintains good performance by preventing overfitting.

Cross-validation hyperparameter tuning is an important component that the paper highlights. XGBoost has parameters that can affect the complexity of the model along with performance, for instance, tree depth, learning rate, and the quantity of estimators.Additionally, careful tuning of these parameters allows the training procedure to generate a more efficient and robust classifier. Generally, the objective of this branch dissects the tradeoff between effectiveness and feasibility of deployment, basically because of the nature of a QR security system which is required to function appropriately under the different conditions of the external world.

3.8 ResNet-18 Tampering Detection Model

As a tampering detector, the image analysis branch employs ResNet-18, further refined on the Binary QR Codes Dataset containing both undistorted and distorted QR images. This branch's objective is to differentiate QR codes that are structurally manipulated,

be it QR code overlays, distortion, occlusion, and/or other QR code anomalies. The input QR code patch is resized to 224x224 pixels and may be converted to grayscale and duplicated as three channels as a means to CNN comply with the expected 3-channel input.

The selection of ResNet-18 for this project is due to its relatively simple architecture and depth. The architecture of the ResNet-18 consists of a residual inner block which is useful for the extraction of deeper and more complex visual representations. QR tampering detection also requires networks that are capable of detecting subtle differences in structures as opposed to just the presence of large visual differences. A strong feature extraction network is therefore deemed necessary. ResNet-18, on the other hand, outperformed the other simplistic CNN-based architecture image classifiers such as Simple CNN and MobileNetV2 in the same tampering detection task. Hence it is the final choice for the tamper detection module.

The model's niche is elucidating evidence on the image level instead of the decoded message. This is crucial because although the image might initially look harmless, a QR code can still be tampered with, and the image branch should be able to detect those structural irregularities through the pixel arrangement. In this respect, a URL classifier and a ResNet-18 model are complementary tools for the QR security challenge.

3.9 Training Procedure for the Deep Learning Branch

The necessary QR image model has observed deep learning training routines. Image resizing is performed to a predetermined size, and training-time augmentation is performed to mimic realistic capture conditions. The paper mentions several augmentations: random rotation, perspective warp, Gaussian blur, motion blur, JPEG compression, brightness/contrast jitters and random occlusion. Real QR images are almost never ideal. They are often taken at oblique angles, in suboptimal lighting and with signal noise and distortion. By training with real sample variations, the model is robust against suboptimal signal images.

The paper states classifications are performed with Cross-Entropy loss and the training routine uses Adam optimizer with an early stopping mechanism based on validation loss. Using early stopping is reasonable as it prevents the model from overtraining

when validation performance decreases. The training routine uses transfer learning with ImageNet pre-train weight which improves generalization and speeds up convergence. This is a reasonable choice as QR image signal datasets are normally much smaller than general-purpose image datasets, therefore the use of pre-trained weights for model initialization is a satisfactory choice.

According to the paper, ResNet-18 showed greater performance in the QR image classification task than the Simple CNN and MobileNetV2. Thus, the selection of ResNet-18 for the image branch in QRShield is justified. The aim of the training procedure is to minimize the loss of model stability and lap the residuals (the difference between what the model predicted and what the desired outcome was) so that the model is robust and provides reasonable accuracy even if the QR images that the model is exposed to differ significantly from the QR images that it has been trained on. This is necessary to ensure the effectiveness of the system during real world applications and not only in synthetic laboratory repeatable experiments.

3.10 Fusion Strategy

The adaptive fusion strategy introduced by the QRShield system integrates machine learning and deep learning to analyze security concerning QR codes and how the QR codes can be further secured. When designing the QRShield system, it was essential to understand that QR codes need to be viewed beyond a single dimension. QR codes consist of a visual design and an encoded URL. These aspects of QR codes can be exploited to facilitate an attack. Given this understanding, QRShield analyzes both the QR code image and the URL it carries. Within the proposed system, the deep learning module specializes in examining QR code images and detecting visual distortions and QR code image integrity. Conversely, the machine learning module focuses on the encoded URL and classifies it as secure or potentially malicious. The system's dependability and confidence to detect attacks that are potentially concealed in the URL will increase by integrating both modules of the proposed system.

First, the Deep Learning analysis using the ResNet18 model is performed whenever a QR image is uploaded to the system. The model analyzes the image data of QR codes to determine if the QR image has been tampered with. The model is able to recognize if

the QR code has been stretched, blurred, partially covered, or has a pattern overlayed, among other visual alterations. The output of the model is a distortion probability value ($\mathrm{dl}_prob).IftheQRcodeislikelytohavebeentamperedwithorhasasuspiciousappearance,thenthevalueish

At the same time, the system attempts to decode the QR code using various decoding and recovery techniques. This includes Pyzbar and OpenCV QR detection as well as several image enhancement techniques. Image enhancements may include denoising, rotation, sharpening, thresholding, and more. The combination of the various techniques described improves the ability of the system to recover QR codes and information that has been lost or obstructed to the point that the QR code and information are not clearly visible or readable. In real-world scenarios, QR codes are scanned with inadequate lighting, at oblique angles, or printed on surfaces that are far from being perfectly clear. Because of these real-world scanning scenarios, it is very likely that the system may need to employ the multiple decoding techniques to improve the overall likelihood of obtaining the URL that is embedded in the QR code.

If a URL is extracted, the Machine Learning module then uses an XGBoost classifier. This branch of the QRShield system uses a different approach compared to other branches. Other branches, for example, may analyze the URL and text string. This branch analyzes the length of the URL, the presence of suspicious/undesirable words, the presence of HTTPS, DNS and WHOIS information, the number of redirects the URL has gone through, and the age of the domain. Analyzing these factors is important. Malicious URLs often exhibit abnormal patterns in one or more of these areas. For example, a malicious URL might be a newly formed domain that contains a suspicious naming pattern, redirects excessively, or contains weak trust indicators. Based on these features, the XGBoost classifier builds a maliciousness score, referred to as $\mathrm{ml}_prob, ametrictoexplainhowphishingorhowunsafetheURLislikelytobe.$

In QRShield, the most important part of the system is the fusion level. Unlike existing systems, QRShield incorporates an adaptive weighting system depending on how likely a URL is malicious based on the Machine Learning algorithm. Not every QR Code should be treated the same. The URL might contain a Safe URL, but the QR Code might contain a URL that looks very suspicious or malicious, but a Safe URL might look suspicious. Because of this, adaptive fusion is used.

If the URL is mostly safe (e.g., $\mathrm{ml}_prob < 0.30), thesystemusestheMLresult95\%ofthetimeandtheDLres$

If the URL is suspicious (e.g., 0.30 $\text{ml}_prob < 0.70), then the system reverses the influence. In this case, 20$

When URLs are almost certainly malicious, i.e., those with a machine learning (ML) probability of 0.70 or higher, the system again weights the ML result at 95 percent. In this case, the URL is clearly dangerous. A QR code may look harmless, but can still be malicious. The QR code may even look legitimate, but the final destination is harmful, and the system cannot let a QR code that looks harmless to the user override the clearly dangerous URL. QRShield makes sure that the system does not underestimate content-based threats. By taking a strong stance on a clearly malicious URL, QRShield ensures that content-based threats will not be underestimated.

The final fusion score is the result of the combined ML and deep learning (DL) probabilities. After a URL threat and QR image threat have been assessed and fused, a score is generated and enters the decision layer which terminates at classifying the QR code into one of the three categories (Safe, Suspicious, Malicious). The Safe category score is at or below 0.30, the Suspicious category score is in the range (0.30, 0.60], and the Malicious category score is above 0.60. Among QR codes, many do not fall into just safe or unsafe. This is a decidedly less useful view of the world. The Suspicious category accounts for the probabilistic nature of the score and end user decision, making the most practical contribution to security. We hope this will be useful in actual implementations of security.

The decision layer transforms the combined probability score into a formal security label. Users can struggle to interpret a score, but with a security label, users can quickly understand the probability of the QR code of being malicious, and can act appropriately. A SAFE result indicates there is not enough evidence of a QR code posing a risk. A SUSPICIOUS result indicates there is evidence of risk, but not enough to be considered fully malicious. A MALICIOUS result indicates there is enough evidence of a QR code posing risk. This output helps the users of the system make better scanning decisions.

To improve robustness, the system considers special cases. If the QR code includes a URL that is shortened, such as a Bit.ly or TinyURL link, the system skips the Machine Learning analysis if the final destination of the link is hidden. Shortened URLs can be risky to users because of the lack of evidence and the concealment of the final website to the user. In these scenarios, the system uses a Deep Learning analysis and creates a suspicious mark. It is a reasonable approach, because the hidden destination cannot

be evaluated before the QR Code is expanded. Because of the concealment of the final website, the system will remain in a state of suspicion towards shortened URLs.

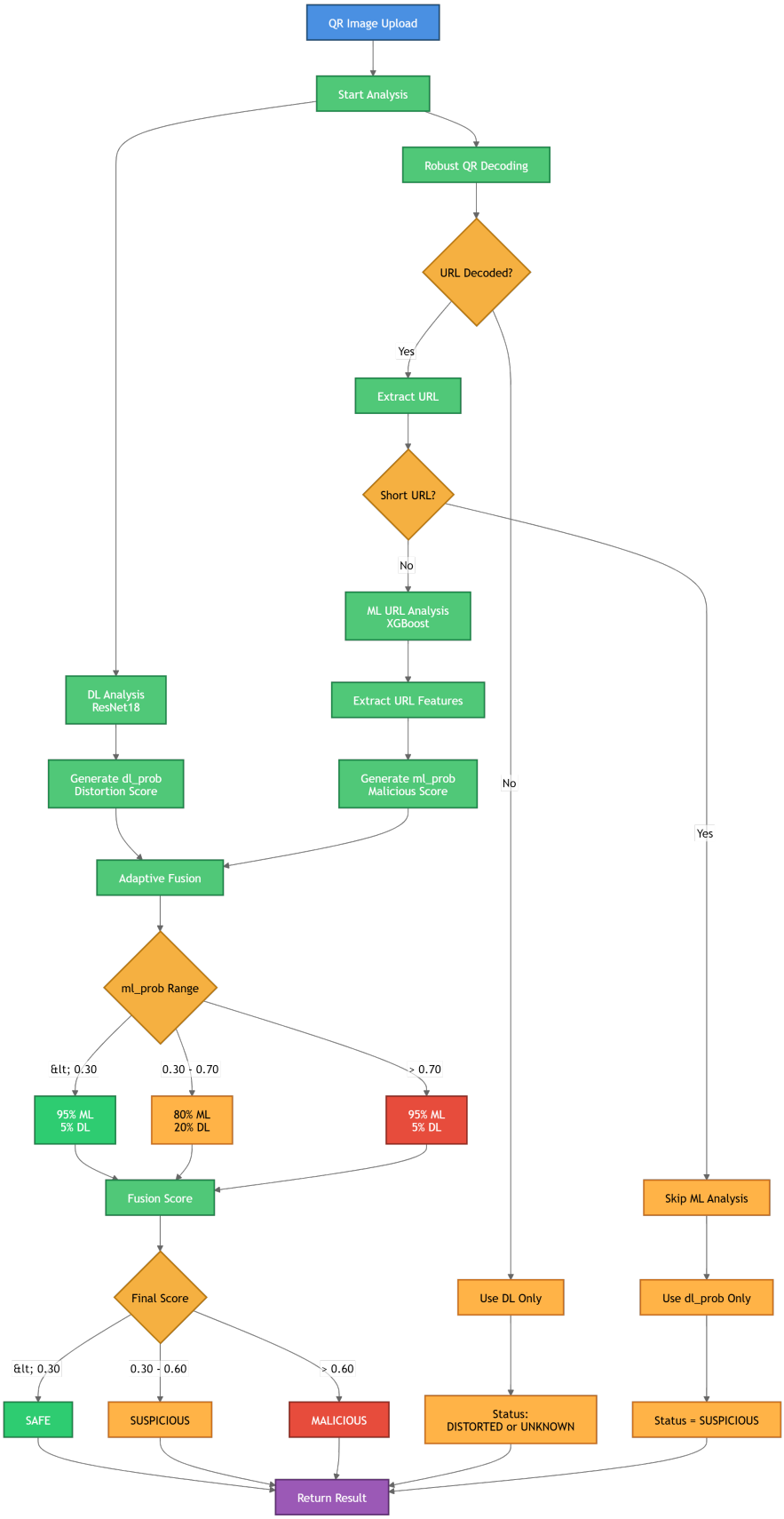


Figure 3.2: Fusion Strategy of the proposed QRShield System

It's the same if the QR code can't be decoded. The system still uses the Deep Learning model to detect whether QR code images are manipulated or simply distorted. This helps ensure the framework generates some relevant output, even when the URL is irretrievable. Normally, QR images are somewhat damaged, blurry, or designed in a way that they deliberately resist being decoded. If the system entirely stopped working in such cases, that would be a major problem. Enemies would take advantage of that defect. Because the image model continues to function when decoding fails, QRShield is stronger.

These exceptions are needed so that attackers can't circumvent the defenses just by hiding the URL or damaging the code. It is designed in a way that it doesn't depend on a single perfect input. It recognizes the quality of information provided and uses whatever evidence is available. This framework is more flexible in realistic situations where QR codes can be excessively soiled and may not be fully readable or transparent.

The fusion strategy made adaptive by QRShield combines the best elements of Machine Learning and Deep Learning. With this strategy, Machine Learning delivers exceptional detection of phishing through URLs and Deep Learning, on the other hand, integrates detection of visual tampering and image-based manipulation with added confidence. Adaptive weighting is an advancement over a fixed fusion rule as it makes the system more flexible based on the level of risk collated from the URL. With this, false positives are significantly reduced, the system is able to accurately identify threats, and the system's overall defense against threats is enhanced. In this case, threats are malicious URLs and manipulated QR images. Enhancing security is key to the reliability of QRShield in providing QR code threat detection services.

3.11 Evaluation Metrics

Standard classification metrics to assess correctness and practical utility are used to evaluate QRShield's performance. Due to its modular nature, multi-level assessment is used. URL classification branch assessment uses accuracy, precision, recall, F1 score, and ROC-AUC score. For the image branch, assessment uses metrics of accuracy, precision, recall, and F1 score. Combined, they give a comprehensive assessment on how well each branch detects malicious QR content and how well each branch avoids false classification

of benign samples.

Accuracy is the proportion of true positives, over the total number of sample test cases. Despite its usefulness as a summary metric, it can give the wrong impression to an imbalanced dataset. Particularly when data is user-generated and especially when focused on security, a model may achieve high accuracy simply by predicting the majority class. For this reason, metrics of precision, recall, and F1 score are especially important to assess performance. Precision is the number of cases the system predicted as malicious, over the number of cases determined as malicious. For a system generating few to no false positives, it is indicative of how few alarms the system generates. In security-focused applications, recall is the most important. It signifies cases of malicious content that the system predicted over the total malicious content. The F1 score is an average of the balance between precision and recall.

For the URL branch, ROC-AUC is reported since it benchmarks the model's capability to distinguish benign from malicious URLs at different decision thresholds. This takes on added importance when the model outputs scores as probabilities instead of hard labels. A high ROC-AUC indicates that the model, in the context of a decision threshold that is likely to be defined based on the deployment context, is effective at ranking malicious URLs higher than benign URLs.

For the QR tampering detector, the effectiveness of the model in identifying manipulated QR patterns is evaluated using accuracy, precision, recall, and F1-score. Since this branch is concerned with visual integrity, it is crucial to understand how the network responds to structural distortions, overlays, replacements, and all other forms of tampering. Regardless of the URL branch, recall is critical because a tampered QR might circumvent a malicious QR payload that goes undetected.

To complement this evaluation, the confusion matrix is highlighted. This showcases the results of true positives, true negatives, false positives, and false negatives, and essentially focuses on the errors that the system produces. Within QR security, false negatives are physically harmful, as these would be classified as safe a malicious QR code. Conversely, too many false positives would render the system unusable, as the majority of legitimate QR codes would be shown as suspicious. For this reason, the analysis of the confusion matrix is a useful method to determine whether the system is balanced sufficiently for a real deployment.

At the stage of performance evaluation beyond classification accuracy, the literature review broaches the subject of metrics related to the implementation of the system. End-to-end latency is defined as the total amount of time the system takes to process the QR code from the moment it is input to the moment the system comes to a final decision. This is a critical metric when the system is employed on mobile devices that perform scanning as part of a real time security mechanism. Memory footprint indicates the amount of RAM, as well as the total storage the system operates on. This metric is critical in determining the system's deployment on mobile devices that have constrained resources. Throughput is defined as the total number of QR codes the system is able to process in a given time frame. This metric is especially important when the system is deployed in a high volume scanning scenario. These metrics, together with the classification scores, indicate the accuracy, efficiency and ease of deployment of the framework.

It is only by using these evaluation metrics that a comprehensive assessment of QRShield can be made. These metrics incorporate a machine learning viewpoint as well as a practical perspective of cybersecurity. This is of paramount importance in assessing the potential system's applicability in practice.

3.12 Summary

This chapter presented the methodology of QRShield in detail. This framework begins by identifying and decoding QR codes, followed by feature engineering of URLs and classification of malicious URLs using XGBoost classification model. The QR image is also passed to a tampering detector based on the ResNet-18 network to identify any visual structure changes of the QR image are found. Then, the outputs of these two branches are fused together to yield the final security decision using a fusion strategy.

QRShield's design is based around the notion that, answering the question of QR security, a single evidence source is insufficient. Semantic information about the destination is included in encoded URL; QR image may have visual tampering/replacement attacks. These two signals are complementary; they can identify threats that may not be found with the use of a URL-based or image-based approach alone. The dual-branch design thus helps to make the design more robust, reduces blind spots, and makes it more

suitable for practical use in security.

The chapter also included the evaluation criteria for evaluation of the performance of the system. Common classification metrics like accuracy, precision, recall, F1-score, and ROC-AUC were added as well as the analysis of the confusion matrix and metrics for deployment like latency, memory consumption and throughput. These are measures that consider the overall quality and/or feasibility of the predictions.

In summary, QRShield is a security system that is layered and defensive, with the goal of protecting QR codes. It is based on a decision making, image analysis and machine learning technology that adds greater protection against malicious QR codes. From this, the experimental setup, results and discussion of the proposed approach is provided in the next chapter.

Chapter 4

System Implementation and Deployment

4.1 Introduction

The chapter outlines the introduction of the QRShield system. The framework provides QR security across varied operating systems and a backend shared by a web and mobile application. The architecture allows for the reception of a QR image from the client, detection of the QR region, decoding, and subsequently analyzing the embedded URL, the QR image, and making a decision by fusing the outputs.

Modularity and scalability were guiding principles of the design. Core detection, and analysis were placed on the backend to avoid putting intelligence directly into the applications. As a result, both the web and mobile applications rely on the same analysis engine, thus leading to the same result when presented with the same QR code. Maintenance was made easier, since the backend processing required in the client applications was minimized, leading to fewer updates on the client applications, as model and pipeline updates were performed on the backend. Balancing the separation of UI and backend processing improved development and maintenance of the system at a later point.

4.2 Overall Implementation Strategy

QRShield applies a client–server architecture. The client includes a mobile application and a web application, both of which can scan or upload QR codes. The server side includes the detection pipeline, which performs QR localization and QR decoding. It also extracts and classifies URL features, detects URL image forensics, performs explainable AI analysis, and labels risk. This design follows the backend-oriented model as described in the paper. In this model, the QR image, captured by a client application, is sent to the server for processing and analysis.

There are multiple tradeoffs in this design. First and foremost, it keeps the ML and DL elements centralized, making the system easier to maintain and upgrade. Second, it reduces the development of duplicate logic in the different client applications; both the web and the mobile applications operate the same backend pipeline, therefore avoiding different logic in an application. Third, it ensures that both applications classify the same input in the same manner, which ultimately increases both the reliability and the consistency of the system. Fourth, it allows for processing to occur in an asynchronous manner, making the interface more responsive.

There are also additional tradeoffs in this design. Upgrades to the explainable AI analysis, additional models, or additional analysis modules can occur in the backend without necessitating a full redesign of the client application, therefore increasing the flexibility of QRShield and helping it adapt to the rapid pace of the real world.

4.3 Web Application Implementation

Implemented as the browser-based component of QRShield, the web application allows users to upload, or scan, a QR code image and receive a fast security evaluation. This version is useful in testing and demonstration, and it allows a user to check a QR code before physical interaction in a desktop environment. With this version, system developers can check the system while the web application process images directly, via a web browser, without necessitating the setup of an application locally.

When an image is uploaded via the web application, it is dispatched to the backend API for processing. The backend executes QR code detection, decoding, URL feature,

and tampering and explainability assessments. The web application receives the final evaluation and displays it to the user. Based on the evaluations, the final output will be assessed as benign, suspicious, or malicious. Finally, a SHAP explainability assessment will be presented to users for further assistance on the assigned QR code label.

The web interface is designed with usability and transparency in mind. Direct access to classification results and explanations builds confidence and aids in understanding the system. The web interface also serves to demonstrate the QRShield pipeline, showing the process from image submission to the final security verdict. The web interface employs the same backend as the mobile app, showing the same logic, which provides consistent behavior across platforms.

4.4 Mobile Application Implementation

The mobile version of QRShield is designed for field use with real QR scanning capabilities at deployment. Since we often find QR attacks on public display boards, restaurant menus, ad frames, product labels, and things that transport, the mobile app is an easy-to-use interface that people can access every day. A mobile scanner is even more helpful since many people are likely to encounter QR codes and will need to do a security check before they access the link.

The mobile app takes a QR code snap via the built-in camera and makes a security check after sending the image to the QRShield server. The mobile app operates the same as a web app and uses the same detection pipeline as a web app. The mobile app does not incorporate any deep learning or machine learning logic for the security checks. The mobile app is a compact client that snaps the QR code and sends it to the server to make a security check and show the verdict made by the server.

This makes using the QRShield mobile app really easy. The compact mobile app saves battery life and allows the phone to run really smooth. Since the detection logic takes place on the server, the mobile app is fast, efficient, and great to use. The user gets the full QRShield analysis wherever they are using the app, and they get the protection of URL security, scam protection, and system security audits.

As a deployment-oriented interface of the system, the mobile application does not intend to mirror the backend intelligence. Instead, the application is designed to offer backend

intelligence in a user-friendly and mobile manner. For that reason, QRShield can be used in countless scenarios where users need instant defense against harmful QR codes.

4.5 Backend Architecture

The QRShield implementation's core is built around the backend system. The primary functions of the backend system include receiving requests from clients, processing the QR image, executing both branches of the detection system, and generating and returning the final classification. In the approach the authors have formalized, the backend acts as a two-stream pipeline. In the pipeline, the QR image is first localized and cropped. Subsequently, it is decoded and then concurrently processed by the URL analysis and image analysis branches. Integrating both branches of the pipeline allows the backend system to leverage both semantic and visual cues prior to making a final call.

A good design calls for each stage of a pipeline to be independently maintainable. In the backend system, each of the components that build the QR image processing system as well as the URL analysis, the proof of concept of the tampering detection system, and the SHAP-based explainability system are separate. With this design, the cost of maintenance is lower because each of the components can be modified without affecting the entire pipeline. Debugging is also simpler because problems in the system can be traced to a specific stage of the pipeline.

With the design approach used, maintenance and future extensions of the backend system is simple. If a more advanced explainable AI system, a more sophisticated URL analysis, or a better image processing system is available, that design component can be integrated. This design allows for flexible architecture and system improvement. The design adopted also helps keep the code organized, a critical outcome for a complex system that integrates computer vision, machine learning, and deep learning.

In general terms, the backend architecture is meant to function as the intelligence layer of QRShield. It conducts the vital security operations behind the scenes and delivers a single, uniform, and interpretable final verdict to the client applications. This balances the system's ease of deployment with the proposed framework's security goals.

4.6 QR Detection and Decoding Module

The backend pipeline's first stage consists of QR detection and decoding. Upon receiving a QR image from a client, the system detects the QR region and reads its content. As described in the paper, in this stage, the system separates the QR region in the image by applying grayscale conversion, adaptive threshold, and contour finder, and performing finder-pattern detection and perspective correction. Here, it is important to understand that QR codes are typically photographed under unfavorable conditions, such as at an angle, with uneven or insufficient lighting, with shadows, or blur, and with partial obstructions. By performing the above techniques, the system can capture a clearer QR patch that should be suitable for decoding and subsequent processing.

To achieve cropping and perspective correction, OpenCV is employed. This software helps QR codes get properly aligned, which in turn helps ensure a normalized, rather than distorted or rotated, QR patch. The main goal for this stage is to help improve the success of decoding QR codes and to prepare a QR Patch inline with expectations for the image tampering detector.

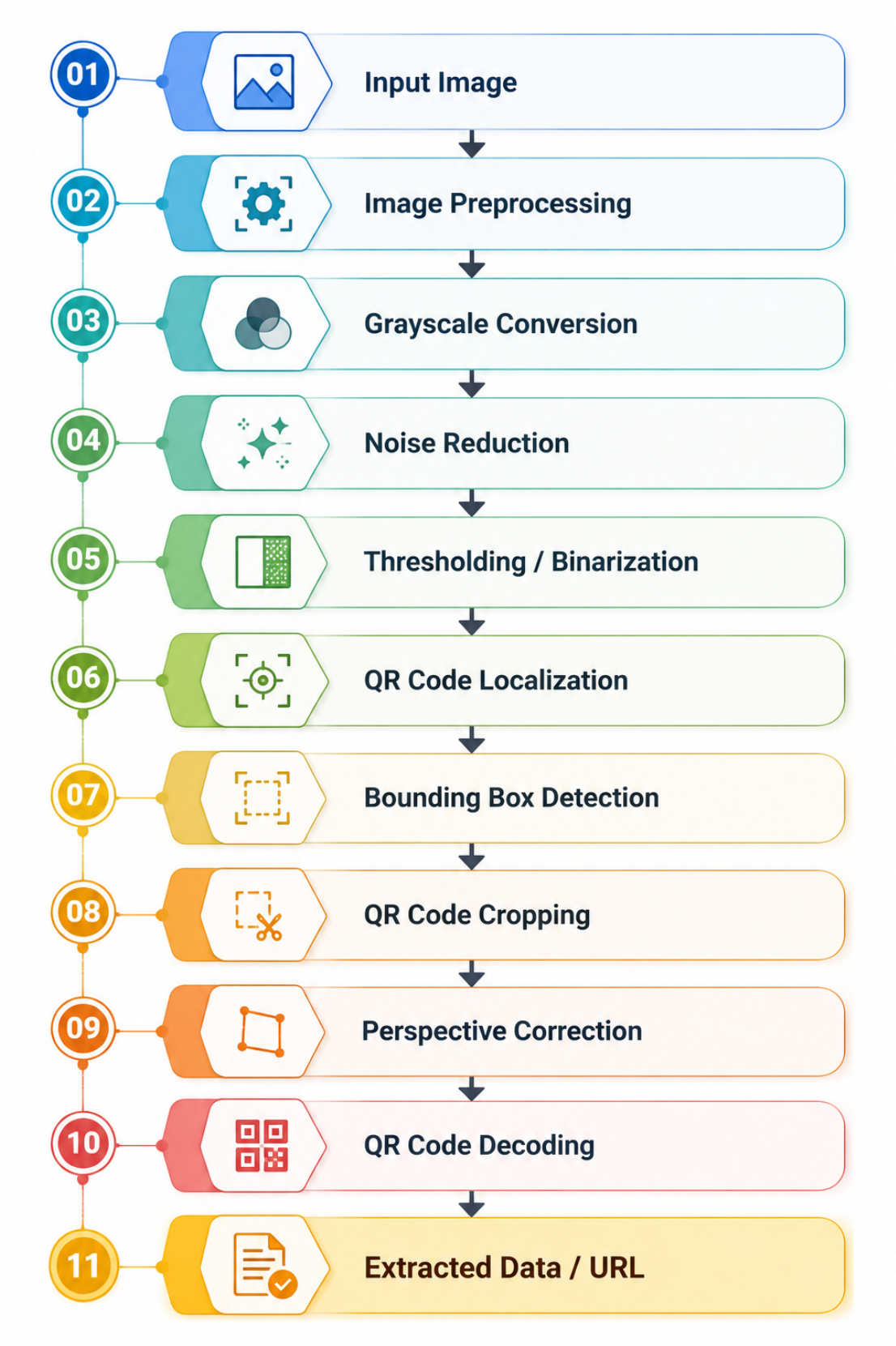


Figure 4.1: Qr Code Detection and Decoding Pipeline

After locating a QR region, the system uses a QR decoder to decode the payload. This can be software such as pyzbar or ZXing. In the case that the initial decoding attempt is unsuccessful, a number of different techniques are performed and the QR decoding is attempted again. QR related images may be enhanced via histogram, equalization, increase in sharpness, morphological closing, and a number of other techniques that improve the interpretability of the QR pattern. The fact that the system has been designed to perform multiple decoding attempts is an attempt to increase the robustness of the system.

This module is extremely important due to the fact that QR codes rarely exist in a lab and they often do not have a clean QR capture. Users scan codes located on posters and screens, on prints and labels, as well as from damaged surfaces or from images of low quality. Many of the examples previously stated may require multiple decoding attempts rather than one. There is a great deal of preprocessing and retrying present in the system. The system will continue security analysis even in the case of low quality images and partial QR capture. In such cases QR payload decoding may not be feasible and the QR patch will be passed on to the QR tampering detector.

The decoding module initializes the backend processing system. It captures QR content and provides a quality QR image for the next steps of backend processing. The module captures a QR payload and a QR image patch so that they may be passed on to the next steps of backend processing.

4.7 URL Analysis Module

In the event that the QR Code is read successfully, the embedded URL is then transmitted to the URL analysis module. This Module is then responsible for the extraction of certain features that are related to security from the URL, as well as the analysis of the features using a machine learning model. The document states that the URL branch employs a representation that is comprised of 27 features, which include some of the lexical, WHOIS, DNS, and HTTP indicators. The focus of these features is on the types of behaviors that can be related to malicious or suspicious links. These would be things such as the length of a URL, the length of a host name, the presence of certain suspicious words, the age of a domain, the presence of an MX Record, the presence of

SSL Certificate, the amount of Redirects, and the HTTP Response.

From an implementation point of view, the feature engineering pipeline is responsible for the construction of certain features, as well as the arrangement of features in a specific order. The construction of certain features is very important for the machine learning model to be able to create a valid and reliable prediction. This is because the presence of a feature that has a malformed value or is missing will greatly affect the quality of the prediction.

From this module, the output is a maliciousness score or a class prediction that is based on, and supported by, the work of the XGBoost machine learning algorithm. The research document states that XGBoost was the most suitable machine learning algorithm and outperformed the other traditional machine learning algorithms, which is why it was selected for the final framework. Therefore, this makes the URL analysis module the most important module of the system's semantic inspection capability, since it is responsible for analyzing the content of the link as well as the behaviors of the link. This is done to determine the activities that the link may be performing in the background, instead of the QR Code itself.

This branch is significant because many malicious QR attacks embed unsafe or misleading URLs in QR codes which otherwise seem normal. A QR code may seem safe, but it may lead to phishing, malicious code downloads, or unsafe websites. The URL module is used to determine these threats. Because the model is built with engineered features, it can identify both behavioral patterns and structure of the text that the module is classifying.

With regards to the entire QRShield system, the URL analysis module is the primary source of the semantic security signals. It adds another layer of protection by evaluating whether the decoded destination URL is safe, unsafe, or malicious. As such, it greatly enhances the system compared to a QR scanner that simply decodes the link.

4.8 QR Tampering Detection Module

Simultaneously, the sectioned QR image goes through the deep learning network to spot manipulation. This uses a modified ResNet-18 model which was trained on the Binary QR Codes Dataset. The aim of this model is to see if the QR image was manipulated

in a way such as distortion, an overlay, occlusion (i.e. covering some of the QR code), replacement, or other structural modifications. These manipulations can be done in order to conceal the actual endpoint of the QR code, or to trick the user to scan a different (and probably a malicious) code.

In the process, the QR patch is scaled to the input size of the model which the paper says is 224 x 224 pixels. During the model's training, various augmentation techniques were used to improve the model's ability to detect QR codes as naturally occurring QR codes are not always clear. Some of the augmentations were rotation, add a perspective warp, Gaussian blur, adjusted the image (and/or its background) by applying compression, and/or adding/removing occlusions. Adding such training data to the model while training improves the model's ability to detect tampered QR codes of various qualities.

This branch provides a result consisting of a confidence score indicating whether the QR image is tampered with. This score shows the model's interpretation of the QR code's structural completeness. The paper reports that ResNet-18 performed better than the other image models which were tested (i.e. Simple CNN, MobileNetV2) and as such was selected for the final implementation.

This module is crucial because not all QR threats are apparent just by looking at the URL. A bad actor could change the original QR sticker by either overlaying a fake pattern or replacing part of the code to modify the encoded contents to something else. In such a case, the user wouldn't be able to tell. The tampering detector detects such attacks by analyzing the QR image. It complements the URL classifier and acts as a second line of defense.

This branch further adds to the URL classifier's capability to defend against attacks beyond URL inspection by analyzing the QR image directly. Additionally, this module helps the system defend against attacks that require physical alterations and visual deceptions of the QR code.

4.9 SHAP-Based Explainability Module

For added transparency, QRShield offers a SHAP-based explainability module for its machine learning branch. Once XGBoost classifies the URL, SHAP values analyze each feature and how it impacts the prediction. This helps the system identify which feature

heavily influenced the classification of the URL.

SHAP's goal for QRShield is to implement explainability into the system and reduce the black box nature of machine learning. This is crucial in a security-based application since users and security analysts need to understand the reason behind any alerts or warnings. Simply putting a label for each prediction is not enough for the user to determine the credibility of the security warning. SHAP improves the explainability of the system by breaking down the prediction and showing how individual features influenced it the most.

Table 4.1: SHAP Feature Interpretation Example

Feature	**SHAP Contribution**	**Impact on Prediction**
suspicious_words	+0.32	Increased malicious probability
domain_age_days	-0.21	Reduced malicious probability
redirect_count	+0.28	Increased malicious probability
ssl_valid	-0.19	Reduced malicious probability

As listed as an example, domain age, number of redirects, count of words labeled suspicious, SSL validation, and DNS behavior are all features that can positively or negatively impact the score assigned to a URL for its maliciousness. If the prediction was a malicious label, the explanation would show how features related to high-risk were present. If no features or low-risk features were present, the explanation would show how the features supported a URL to be classified as benign. Then, the overall classification of the URL would be supported by the explanation.

SHAP explanations in the web application can show detailed visualizations in the form of feature contribution bars, ranked views of contribution importances, and/or contribution plots for each prediction. This type of detail provides model reasoning to analysts, researchers, and other users who understand this level of detail and want to see the model reasoning. The mobile application simplifies these details and presents them in a

concise summary so that users can access the main reason for the alert. This solution is optimal for expert and non-expert users.

The SHAP module is run after the XGBoost prediction and before the final result is produced. As a result, QRShield does not generate a label, but provides an example of how to support trust and interpretability of the model for users who want to understand and reason about the prediction.

The interpretability of the framework is highly welcomed and it is especially useful in security-related analysis as it helps users understand the grounds of the classification decisions.

4.10 Fusion and Final Decision

In QRShield, the final classification is created by integrating the outputs of both the URL and image branches. The fusion step of the classification process is critical in the QRShield system. By integrating semantic information from the URL and visual information from the QR image, the framework can make an informed security decision. As malicious QR codes can be dangerous from either the destination link or be physically altered, relying on only one branch makes the security system vulnerable. By integrating outputs of both branches, QRShield is designed to be more robust and usable in practical scenarios.

This paper presents two fusion strategies, rule-based fusion, and score-based fusion. In rule-based fusion, the detection of malicious URLs takes precedence. Therefore, if the URL branch output is classified as a malicious link, then the QR code is classified as malicious, regardless of the image branch output. The detection of tampering, especially in combination with URL behavioral suspicion, can also increase the overall risk level classification. In security, the cost of missing a threat is high, and therefore, the rule-based fusion approach is conservative. In the case where a QR code is visually altered and the URL is somewhat ambiguous in its maliciousness, the QR code will be classified as suspicious, not completely safe. This ensures that the system is in a constant state of alert.

Score-based fusion normalizes and merges probability scores using a weighted decision process or a small meta-classifier. This allows the system to take into account both the

final hard label and the decision confidence of each branch. For example, a final risk score may be higher than either score when a strongly indicative score of tampering and a moderately suspicious score of a URL are fused. If one score is uncertain and the other score is firm, the fusion layer can optimize the decision. With score-based fusion, more patterns are captured and improved when compared to fixed rules, because of the increased flexibility of the fusion.

The implemented system's fusion layer gives the output of benign, suspicious, or malicious. This three-class system is helpful because not all QR codes can be classified as completely safe or unsafe. Suspicious QR codes can show visual modifications, and their URL behavior can be unclear and not strictly malicious. The three-class system captures the system's concern, where the system believes something may be suspicious, and thus represents the system's concern in an output that is practical. The output is then communicated either via the web or mobile application to the users, along with the SHAP explanation of the URL branch.

The benefit of this design is that the system utilizes both the content and visual sides of the QR code before determining safety. It makes the final decision more trustworthy than a single-modality scanner that only verifies the URL or only analyzes the image. In security-related situations, this system of multiple layers of evidence is useful because attackers may present threats in different forms. QRShield solves this problem by merging corroborative evidence into one conclusive answer.

4.11 Deployment of Web and Mobile Clients

The QRShield system consists of a mobile application and a web application that both work together and connect to the same backend. It is necessary to keep the security logic identical across both platforms because the same QR code should come to the same conclusion no matter if it is being scanned from a desktop browser or a mobile phone camera. By connecting both clients to the same backend, the system eliminates the inconsistencies that may arise from having each client work with a different local model or analysis pipeline.

The web application is ideal for desktop testing and demonstration purposes. It has a user-friendly interface that allows the upload of QR images, and it shows the results of

classification for the users. It is useful for evaluation and development of the system and for its presentation. Users can see the results and explanation of the classification without the need of the mobile application, and this is useful for testing the system and for debugging.

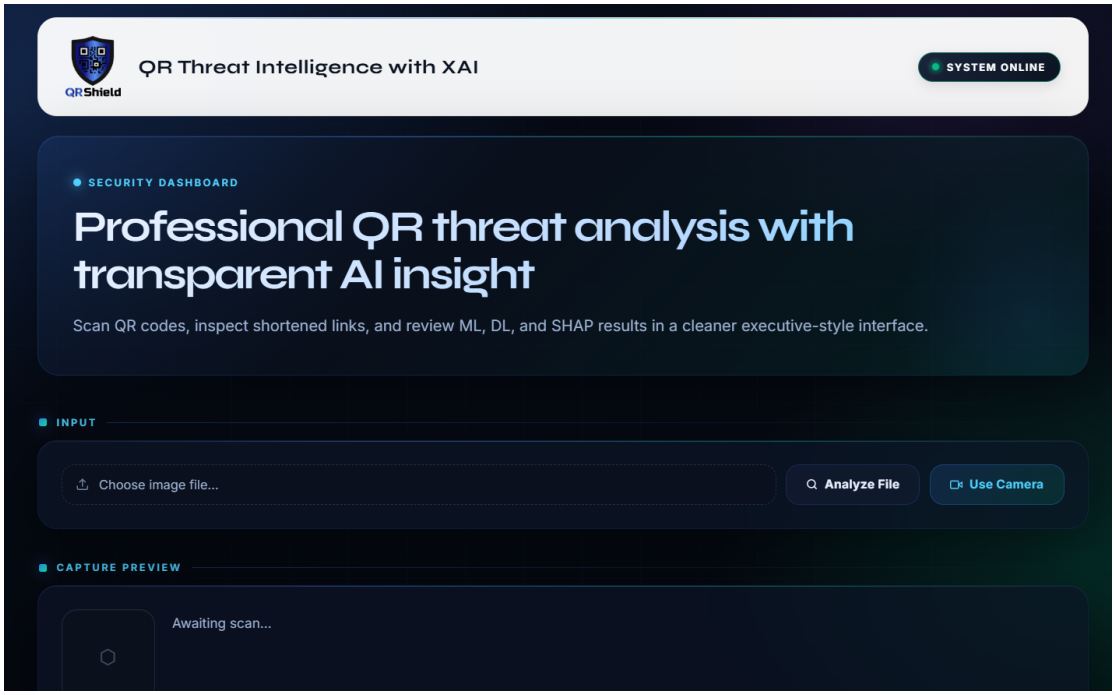


Figure 4.2: QrShield Pro Website Interface 1



Figure 4.3: QrShield Pro Website Interface 2

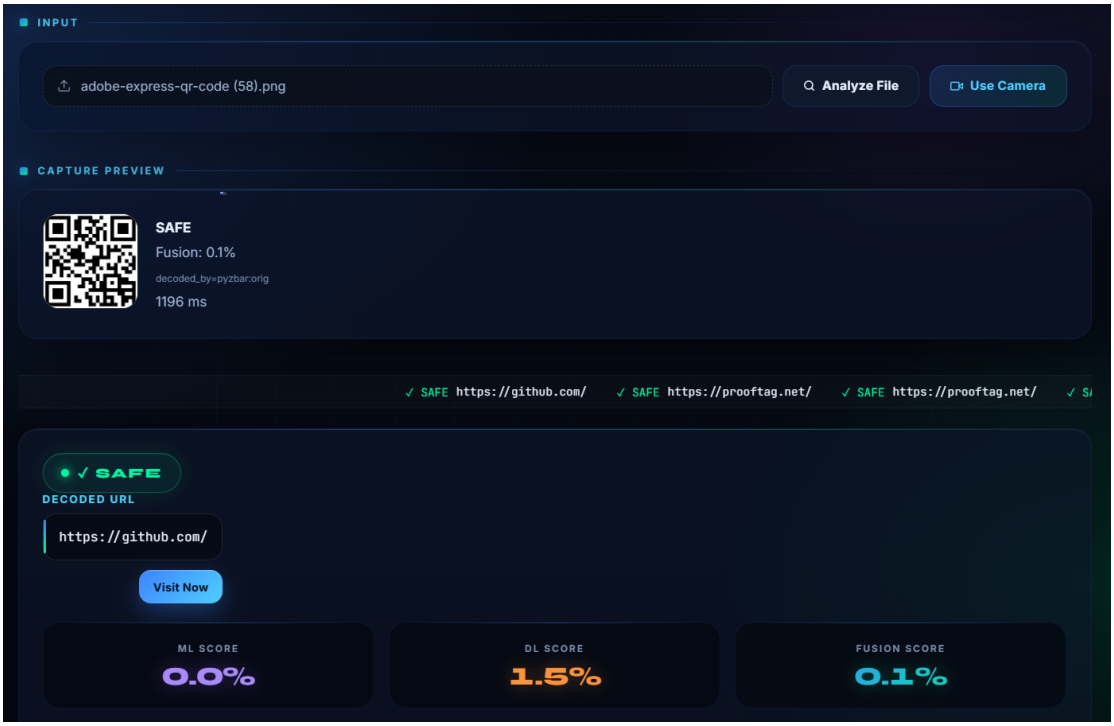


Figure 4.4: QrShield Pro Website Interface 3

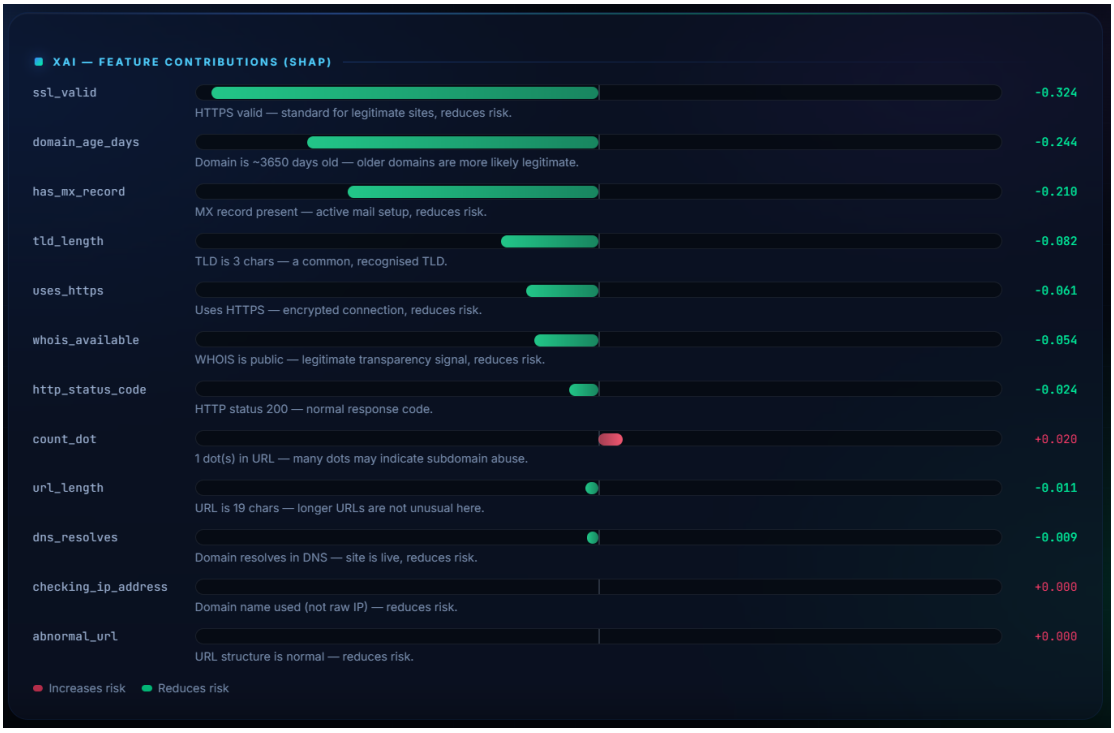


Figure 4.5: QrShield Pro Website Interface 4

The mobile application of the system has the highest value when performing analysis in

the real world because QR attacks take place in the real world as well, and they are often found on posters, menus, advertisements, payment terminals, and packaging. Since the QR codes can be found in a huge variety of places, the mobile application allows users to quickly perform security analysis of the QR codes. Most of the time, users perform security checks on QR codes just before they open a potentially harmful link, making mobile application the best option to offer security in everyday instances.

Deployment on the Play Store indicates that the project has transitioned from a prototype to a working security application. This also suggests that the system's design incorporates working solutions and is not confined to a lab environment. The existence of a working mobile client also validates the framework by showing that the QRShield analysis pipeline is capable of supporting real world mobile user engagement. The web client is still useful for testing, demonstration, and engagement purposes.

The integration of both clients widens the QRShield's application in different contexts. The desktop accessibility of the web interface, together with the mobile interface, establishes a compact system for security verification and analysis that can be accessed from anywhere. Because both clients rely on the same backend system, they are a single, integrated system.

4.12 Implementation Benefits

This strategy offers a number of advantages. First, the front ends stay user-friendly and easy to use. Clients deal with image analysis, upload, and display. They remain simple. This responsiveness is necessary for a mobile experience when users prefer a system that works with minimal input. Because the back ends do the bulk of the analysis, client applications do not need to perform any resource-consuming analysis

The architecture places models at the back end, so improvements can be made to backend services without need for client updates. Maintenance of the classifier, the detector, and the module(s) is done at a single point. The architecture extends the system without requiring mobile and web client updates. The architecture also serves to benefit all the users with the latest detection capabilities.

The system is flexible to accommodate rapid improvement of the various steps in the processing. Improvement of any one of the modules can happen with minimal inter-

ference to the rest. The steps of the design include analysis of the URL, assessment of the integrity of the system, and interpretability among others. The system can be extended with in depth analyses of the same system. This flexibility will be beneficial in the research phases and also in subsequent system deployments.

Fourth, the system keeps consistency across web and mobile environments, important for usability and reliability. QRShield ensures that the final result is based on the same logic across platforms. This is helpful as users will avoid confusion and have a sense of dependability for the system. A user should not receive a different safety judgment based on the interface they used to scan the QR code.

Another important advantage is that the system uses parallel processing. As stated in the paper, the URL analysis branch and the image analysis branch can both function at the same time. The system can now process the both branches at the same time, requiring no wait time for sample to finish processing. This also heightens the efficiency of the system, as the speed of the overall process is dependent on the slower of the two branches. The use of parallel processing is helpful, as it serves the overall purpose of the system: to allow for the quick and efficient scanning of links.

The use of efficiency and consistency in the framework of QRShield is also helpful, as it fulfills the purpose of the system for both a research tool and for a practical tool.

4.13 Summary

This chapter provided details about implementing QRShield as a cross-platform client-server system with web and mobile interfaces. The backend of QRShield processes the QR image, identifies and interprets the QR code, retrieves the URL, establishes 27 security feature evaluations, utilizes XGBoost for URL classification, employs ResNet-18 for QR image evaluation and SHAP for explanatory generation, and merges the outputs to create a labeled QR code and provide the final output. The design of the proposed system is modular, scalable, and explainable and is suitable for real-life applications. The next chapter will provide the results and discussions of the implemented system.

Chapter 5

Results and Discussion

5.1 Introduction

This chapter provides details of the experiments carried out from the proposed QRShield framework, followed by an analysis of the framework's machine learning and deep learning components. The assessment involved two datasets created by the team: Malicious URLs Features Dataset for URL classification and Binary QR Codes Dataset for visual tampering detection. The two datasets were developed to investigate the two main security branches, where one is concerned with the embedded URL, and the other is focused on the visual integrity of the QR image.

The framework aims to detect both malicious payloads and tampered QR images because attacks via QR Codes can occur on both the semantic and visual sides. A QR Code can have a harmful link and still appear visually 'normal,' and a QR Code can appear visually 'normal' and still behave in a harmful way. The outcomes indicate that URL analysis and image analysis should be conducted in tandem, and that one of the two inspections should not be conducted in isolation.

The assessment employed standard metrics including accuracy, precision, recall, and F1-score. Since both false positives and false negatives have an impact on security, these metrics provide a balanced view of model performance. For the URL branch, the proposed framework evaluated various machine learning algorithms to detect the best model for developing a structured feature-based detection. In the image branch, multiple deep learning models were evaluated to recognize tampered QR images. The

paper states that for URL detection, XGBoost outperformed all others, and for QR tampering detection, the best model was identified as ResNet-18. Thus, these findings justify the design decisions made for the QRShield framework.

5.2 URL Classification Results

The URL analysis module used a 27-feature representation derived from decoded QR URLs. These features drew from lexical, WHOIS, DNS, and HTTP indicators, and encapsulated the structure of the URL and the behavior of the associated domain. Such representation of a feature set is appropriate for malicious URL detection. Malicious URLs tend to exhibit patterns in the structure, registration, resolution, and HTTP response of the URL.

The results indicate that traditional machine learning algorithms performed well on this type of structured data, with XGBoost yielding the best results. This performance indicates that the synthetic features allowed traditional algorithms to formulate good decision boundaries to classify URLs as benign or malicious. Since the data was in a processed and meaningful feature space, the classifier could work on the relationships and processed features as opposed to the raw URL and its contents.

Table 5.1: URL Classification Performance Comparison

Model	**Accuracy**	**Precision**	**Recall**	**F1-Score**
Logistic Regression	0.97	0.97	0.97	0.97
SVM	0.98	0.99	0.97	0.98
Random Forest	0.985	1.00	0.97	0.98
XGBoost	0.99	1.00	0.98	0.99

The findings in the paper show that Logistic Regression has an accuracy of 0.97 with matching scores of 0.97 for both precision and recall and an F1-score of 0.97. The Support Vector Machine achieved an accuracy of 0.98 with precision of 0.99, recall of 0.97, and an F1-score of 0.98. Random Forest had a score of 0.985 in accuracy, 1.00 in precision, and 0.97 in recall and F1-score of 0.98. The best performer, XGBoost, achieved 0.99 in accuracy, 1.00 in precision, 0.98 in recall and an F1-score of 0.99.

These observations are a strong indication that the URL features engineered from the dataset were quite effective and that the classifier was capable of detecting harmful URL with great accuracy.

In a security context, XGBoost was excellent in both precision and accuracy as scoring of 1.00 in both measures indicates that the model had no false positives. In practice, the model would be of little value if many false alerts were generated as users would lose faith. A model exhibiting excellent recall and precision is, therefore, a great asset. The paper also cites the heterogeneous nature of the features employed by the model as beneficial, particularly the infrastructure-related features of the domains and their trust value and/or risk value as indicated by the presence of an MX record and secure communication.

5.3 Visual Tampering Detection Results

The second part of the system dealt with the detection of visual manipulation of QR codes. For this, the Binary QR Codes Dataset was used, which contains both clean QR codes and distorted QR codes. The test set consisted of 90 images, 40 of which were clean, and the remaining 50 were distorted. To ensure the model's robustness and ability to capture QR codes in the real world, image augmentation was performed. The aim of this capture was to check whether the QR image has been manipulated through one of the distortions, overlays, warping, occlusions, or other structural changes. This is important because QR code attacks may involve physically changing the QR code while keeping the payload hidden. Such a mechanism would provide a supplementary defense against URL analysis.

Table 5.2: QR Image Classification Performance Comparison

Model	**Accuracy**	**Precision**	**Recall**	**F1-Score**
Simple CNN	0.9556	1.00	0.92	0.9583
MobileNetV2	0.9778	1.00	0.96	0.9796
ResNet-18	0.9889	1.00	0.98	0.9899

In this case, the performance of three different models (Simple CNN, MobileNetV2, and

ResNet-18) was compared. Based on these metrics, the Simple CNN model achieved an accuracy of 0.9556, a precision score of 1, a recall score of 0.92, and an F1 score of 0.9583. MobileNetV2 achieved an accuracy of 0.9778, a precision score of 1, a recall score of 0.96, and an F1 score of 0.9796. The best results were achieved by ResNet-18, with an accuracy of 0.9889, a precision score of 1, a recall score of 0.98, and an F1 score of 0.9899. Based on these results, ResNet-18 was the most suitable architecture for the detection of QR code tampering in this set.

ResNet-18 being able to learn to distinguish differentiates cues QR structures from distorted/ manipulated ones suggests that residual networks are ideal for this use case. It seems like the model learned to associate numerous forms of overlay distortion effects, warping, and occlusion that alter the internal structure of the QR code. ResNet-18 outperformed the simple CNN and MobileNetV2, while good generalization, the paper notes that the model proved to be stable in training and generalizing further supporting the use of ResNet-18 as the final image-analysis branch within QRShield.

These results are significant. With the use of correct examples, ResNet-18 can easily learn to detect visual tampering, and with the use of augmented images, this branch can even be utilized to identify visual tampering. As a result, this image branch reinforces the functionality of the URL classifier, especially in situations where a QR code image has undergone visual modification or has been physically changed.

5.4 Discussion of the URL Model Results

Clearly, the embedded URL feature set can be used to detect malicious QR l inks. The results of URL classification demonstrate that QR s can be classified as malicious. The enhancement of predictive quality from Logistic Regression to Entropy and from SVM to Random Forest and XGBoost indicates that QR l inks may be of a more complex nature and therefore, more advanced methods of classification may be applied to this particular data set. Simple linear models were a strong starting point, but feature interactions proved to be more advanced and higher-level models. This tells us that complex data models may have a positive impact on the detection of malicious URLs.

In this case, XGBoost's good performance entailed a combination of a lexical feature and WHOIS, DNS, and HTTP features, among others. Malicious URLs cannot be detected

with one easy feature, and the overall risk from a malicious URL may be composed of multiple disparate pieces of evidence. An apparent malicious URL may be composed of a banal, unregistered domain. However, this URL may have a recent domain registration, be composed of a malicious URL and be confirmed with a malicious SSL. This may be an example of a XGBoost URL classification in the context of an unregistered malicious domain.

The feature assessment in the paper indicates that classification decisions were positively impacted by the presence of domain-related information. Concerning domain age, a presence of an MX record, and HTTPS URL classification, a domain may be considered malicious. This is in alignment with other literature concerning information security, and it indicates that a domain may be classified as malicious if it has a domain registration date, an MX record, and a domain.

To some degree, the URL model is focused on more than just the design structure of the QR Code. Even if the QR Code image looks good, the URL branch can still detect cloaked malicious payloads lurking behind the QR Code. This makes the system more robust to quishing attacks, which hide a malicious endpoint behind a QR Code image. The URL model, therefore, provides an additional layer of semantic analysis to the visual inspection performed by the image branch.

5.5 Discussion of the QR Image Model Results

The results of visual QR image manipulation shows deep learning technique can be effectively used for QR image manipulation detection. Simple CNN, MobileNetV2, and ResNet-18 were compared with each other based on their performance. Simple CNN provides a baseline, but struggled to identify less obvious manipulations. MobileNetV2 and ResNet-18 improved the performance with advanced feature extraction. ResNet-18 gives the best performance and the optimal results, providing evidence that residual learning is a very strong technique to identify QR images that are unaltered as compared to QR images that are altered.

Therefore, each of the three presented models achieved very high results of precision, of QR images being altered. Since the model was conservative, this means that the model was unlikely to alter QR codes that were not altered. As was the case with ResNet-18,

better results for the alteration of QR codes were achieved. Therefore, a better balance between precision and recall was achieved with the final selected model.

The results also support that QR image alteration can be effectively detected if enough representative examples exist in the substrate, augmented, and simulated distortion in ways that QR codes may be captured in the real world (for, example, captured in another way). These distortions can be in the form of changes in focus, changes in tonal values, and blurring, as well as compression. All of the aforementioned examples can enhance the QR detection outside the scope of the design.

The results from the image model indicate that visual inspection is indeed a robust component of the QRShield framework. A QR code could look perfectly fine and still contain manipulation that modifies its meaning and conceals a harmful endpoint. The tampering detector assists in these instances by looking directly at the structure of the image, which makes the system more robust and difficult to circumvent.

5.6 Overall System Interpretation

Considering the combination of results from the experiments, we can conclude that QRShield is efficient because of the two different detection strategies. Detection of malicious destination links is performed by the URL branch with feature-based machine learning. The image branch is used for detecting visual tampering, which is done by deep learning methods. The branches represent answers to different problems, and the results show impressive performance of both branches on their tasks.

The detection of malicious links is performed by QR codes that, at first, seem normal like QR codes but are NOT. The Image branch solves the QR codes that are tampered with visually. With both branches of QRShield, the framework as a whole becomes stronger and ready to face different attacks involving QR codes.

Table 5.3: Fusion-Based Performance Comparison

Detection Method	URL Analysis	Image Analysis	Fusion	Result
URL Only	✓	×	×	Detects malicious links only
Image Only	×	✓	×	Detects visual tampering only
QRShield Hybrid	✓	✓	✓	Detects both malicious URLs and tampered QR codes

The results further indicate that explainability is relevant for security applications. By using the SHAP for the URL branch, the system will be able to display which feature influenced the prediction, hence the model will be easier to interpret. This can be helpful to end users and security analysts, as it will explain why a QR code was flagged as suspicious or malicious. This could enhance user confidence and make the system more applicable to real-world use.

It is also noticed that the results are in favor of the modular design of QRShield. Each of the components can be considered individually and each has a significant contribution to make towards the overall system. The framework is simpler to be enhanced in future work because the whole framework does not need to be changed if a better classifier is used, a new image model is applied, or a bigger feature set is used. The experimental results thus not only confirm the current system, but also the potential for further extension.

5.7 Explainability and SHAP Discussion

A key aspect of it is the explainability for the URL model, which is based on SHAP. SHAP is not only a technical implementation but is also an important part of the interpretation of the results due to its ability to display the influence of the features on the prediction. This is particularly true for security uses, where a final label might not be sufficient. The reason for a URL being flagged as suspicious or malicious is typically the most important thing users and analysts want to know, particularly if it is as to whether the URL should be trusted or blocked.

SHAP is useful to understand in QRShield to help visualize the impact that features like domain age, redirect count, suspicious words, SSL validity, and DNS behavior have on

your site. These features can be helpful as they indicate reliability of the destination and of the domain hosting it. If computed, SHAP values indicate the effect of each feature on the prediction towards a benign or malicious outcome. This allows the model's behavior to be more easily understood and can give insight into the relative significance of the indicators in the decision.

This type of explanation increases the transparency of the system and its trustworthiness. Security: Transparency is useful because in a security context, users sometimes don't trust an automated warning if they don't understand the reason. The uncertainty can be reduced with the help of SHAP as it provides a feature level explanation of the prediction. It also provides insights for debugging and analysis, as developers and researchers can determine if the classifier is using meaningful signals or unexpected patterns.

Explanations can be presented in a more detailed visual form in the web application. These could be feature contribution bars, ranked importance views, or any other visual representation that helps to make the decision making process more understood. If you are a technical user, an analyst, or researcher, you may want to look at these more detailed outputs, as they are useful to you. In the mobile application, the explanation may be simplified so that end users will get a concise, explanatory message without getting lost in "techno jargon". The system has dual presentation, which is helpful to both experts and non-experts.

In summary, SHAP enhances QRShield's capabilities beyond its role as a detection system, making it a more transparent security solution. It helps to build trust, to be more interpretable, and to be more useful for use in the real world by the URL classifier.

5.8 Limitations Observed in the Results

While the results reported are promising, some limitations of the study should be noted. A drawback was that the datasets were team-generated and relatively small especially for the image branch. The results are encouraging but a greater amount and variety of data would provide more evidence of generalization. In this case, the testing on a larger number of QR images from more natural settings, on various devices, and under various attack conditions would strengthen the results of this study with respect to their

effectiveness in real-world situations outside those used in the study.

The other drawback is that the URL feature extraction process relies on external data like WHOIS data, DNS information, or data obtained from the HTTP protocol. The literature review mentions that sometimes these lookups aren't available, aren't immediate, are incomplete, or aren't reliable in live environments. This can cause different features to be extracted in different ways and can adversely impact performance in cases where the information needed for a feature cannot be obtained. In a real system, such a scenario calls for a plan for dealing with missing data and a fallback plan to ensure reliability.

Another constraint is that it is a backend-driven system. This is helpful as it provides a single data intelligence point and lets both web and mobile clients use the same analysis pipeline. But it also requires network connectivity and can create delays for the final response. Slow or no network may impact the user experience. One reason for this trade-off is that QRs are often required to provide instant feedback during the scan.

The paper acknowledges this fact and proposes that deployment related aspects (such as latency, responsiveness, robustness) be taken into account in addition to the classification performance. The security system is not assessed solely through the accuracy of malicious/benign label prediction, that is, a security system is not evaluated just by how well it can predict malicious or benign labels. It should also be efficient in use and be usable in real situations. Future enhancements may need to incorporate aspects beyond model accuracy such as speed, robustness and stability of operation.

Overall, however, these constraints do not detract from the effectiveness and design of the framework. The restrictions do not detract from the major role that the system plays, but they do point to places where further research and testing in larger scales might be helpful.

5.9 Summary

QRShield is effective in malicious URL detection and QR image tampering detection, as shown by the experimental results. XGBoost had the best results for URL classification and ResNet-18 had the best results for image tampering. SHAP makes the system more transparent than single-modality solutions and the final fusion strategy makes the

system more robust than single-modality solutions. In conclusion, the findings align with the proposed thesis, which suggests a combination of machine learning and deep learning can be a viable approach to QR code security.

Chapter 6

Conclusion and Future Work

6.1 Research Summary

This thesis proposed QRshield, a hybrid QR code security framework, which could detect malicious URL and visually tampered QR codes. The impetus for this work was the ubiquitousness of QR codes in digital payments, authentication, marketing and contactless applications, and the rising threat of QR code quishing attacks and QR code visual tamper attacks. The ease of use of QR codes, along with their speed and convenience, has made them an integral part of everyday activity. The same ease of use, though, means that users can easily be tricked by a malicious QR code they scan without knowing the destination first. That's why QR codes are an easy prey for attackers who would like to trick users with malicious links or QR images in an unexpected manner.

The main objective of the system is to overcome the limitations of using only URL-based detection methods, it incorporates both machine learning and deep learning to tackle image tampering detection. The URL branch was represented with 27 features and a XGBoost classifier, and the image branch was represented by a ResNet-18 model that classifies QR images that are clean and distorted. The output of both models was then combined to have one final risk label. The hybrid design enables the system to analyze QR codes from two different viewpoints: content and visual structure. The combination is key because each of the modalities on their own may not show the true danger. A QR code can be visually correct and be linked to a malicious URL, or be altered visually to change the behavior of the code prior to even viewing the code payload.

The work was supported by two datasets created by the team: Malicious URLs Features Dataset and Binary QR Codes Dataset. These data sets were used to facilitate controlled experiments for both arms of the system and allowed the assessment of QR security, both from the payload and visual integrity approach. Using these datasets, the study was able to compare different machine learning and deep learning models in a structured way. The research paper concluded that this hybrid approach is more comprehensive than the other approaches which include only URL or only image. QRShield's unified solution, when used together, creates a comprehensive and realistic security solution to counter today's QR-based threats.

6.2 Contributions of the Study

This study contributes to the QR code security field in various ways. It was among the first to suggest a hybrid mechanism of QR threat detection by using machine learning and Deep learning. This is crucial as dangerous QR codes can be dangerous due to the URL embedded, or because of the way the QR image has been manipulated. QRShield offers enhanced protection as compared to single-layer systems by analyzing both. This is a useful addition as it recognizes that QR threats are multi-faceted, and not necessarily reliably identified through any single type of inspection.

Secondly, the paper introduced a structural representation of QR derived URLs and 27 features. The URL classifier was enhanced with features like lexical, WHOIS, DNS, and HTTP features to facilitate the detection of malicious activities at the surface and infrastructure. This feature design has enabled XGBoost to achieve good performance. This is important since the model can access numerous signals that can mean a URL is trustworthy, suspicious or dangerous. The system represents the text in more detail rather than as plain text, thereby improving the text detection quality.

Third, a QR tampering detector, based on ResNet-18 network, was designed and trained with clean and distorted QR images. This branch was able to detect overlays, distortion and occlusion and other patterns of visual manipulation. The paper claims the ResNet-18 has greater accuracy than the other image models tested, making it a good choice for a final framework. It's not a simple task and it's a proof of a deep learning applied to the analysis of QR image integrity without anything else.

Fourth, the structure comprised decision-level fusion in which the output of both branches was fused at the decision level to make the final classification. This is a practical contribution as it reduces the chances of a false negative in the event that the attack is detected in one modality. The system is more reliable as it doesn't depend on a single signal to make a decision. It helps the system to combine the semantic evidence and visual evidence before final labelling, however.

Finally, the implementation included explainability of the machine learning side using SHAP. This was done to make the model more interpretable, revealing which attributes were used for the prediction, thereby giving both technical and general users greater transparency of the model. In cybersecurity, explainability is crucial as users have to know why a QR code was flagged. The system can be more easily trusted, analysed and debugged by showing the contribution of individual features.

6.3 Achievement of Research Objectives

This research's aims were attained.

The first goal was to implement a hybrid QR security framework combining URL based machine learning with image based deep learning. The QRShield architecture, consisting of XGBoost and ResNet-18, was integrated to a single pipeline to achieve this objective. The system is able to combine semantic analysis of the decoded URL with the visual analysis of the QR image effectively.

The second goal was to find the full range of security features of QR URLs. This was done with the URL representation that uses 27 features, such as lexical, WHOIS, DNS, and HTTP features. These features were sufficient for the classifier to learn and classify benign URL with high accuracy against evil ones.

The third one was to train an XGBoost model to detect malicious URLs. The paper demonstrated that XGBoost achieved the highest accuracy (0.99), precision (1.00), recall (0.98) and F1 score (0.99) among the classical ML models tested. This validates that the goal of the URL analysis branch was achieved and results were very reliable.

The fourth goal was to train a ResNet-18 network to detect QR tampering. This was accomplished and the results presented that ResNet-18 attained 0.9889 accuracy, 1.00 precision, 0.98 recall, and 0.9899 F1-score in QR image classification task. The results

prove that the image branch also worked well and is able to identify tampered QR images with good accuracy.

The fifth objective was to perform a fusion at the decision level of both outputs. This was done using the final QRShield classification layer that fuses URL analysis with image tampering analysis to arrive at a final verdict. The fusion layer assures that multiple sources of evidence are used in the final decision, thereby enhancing the overall security logic.

The sixth objective was to test the system on the teams' own data and with metrics and standards. This was also done using accuracy, precision, recall, F1-score and other assessment metrics for both branches. The evaluation revealed that both models were effective, which validates the proposed hybrid model.

Table 6.1: Research Objectives Achievement Summary

Research Objective	**Status**	**Evidence**
Develop hybrid QR framework	Achieved	QRShield architecture
Detect malicious URLs	Achieved	XGBoost performance
Detect QR tampering	Achieved	ResNet-18 results
Implement fusion strategy	Achieved	Final classification layer
Evaluate system performance	Achieved	Accuracy and F1-score results

Overall, the objectives were aligned with the final system design, and the experimental results confirm that the proposed approach is both effective and practical for QR code security analysis.

6.4 Limitations

The findings were encouraging but some limitations of the study should be recognized. While these restrictions do not detract from the overall impact of the work, they do demonstrate areas of scope for enhancement in the future for research and deployment.

One limitation is the relatively limited size of the datasets. The dataset for the URL was 1000 examples and the QR image dataset was also not a large scale dataset like industrial

datasets. Although these data are adequate for controlled experimentation and model comparison, a greater amount of data would give greater assurances of generalization. Especially, the models would have to learn different kinds of malicious patterns, benign patterns and tampering styles by having more training samples. This is critical because the QR attack in the real world may take many forms, depending on the environment, the attacker's technique, and the QR code being attacked. The more information it has, the more likely it is to be stable and be able to cope with new situations.

Another drawback is a reliance on outside services for the WHOIS and DNS-based functions. These services in real-world applications could be unavailable, slow, inconsistent, and rate limited, impacting feature extraction reliability. The paper also mentions that in practice QR and URL capacities could be limited or incomplete. This is an important issue because these values are crucial to the URL branch to make an accurate prediction. In some cases, if the system fails to retrieve certain network and/or registration information, the feature vector can be partially incomplete, potentially lowering the confidence of the classifiers, or leading into a fallback mechanism. These external dependencies can, therefore, in a live environment, impact the stability and performance of the analysis pipeline.

Another caveat is the current design with backend processing. While this simplifies the web and mobile applications, and centralizes models, it does create network latency and privacy issues. As the QR image has to be uploaded to the server to be analysed, it is important that the internet connection and server responsiveness of the QR is considered. The network could be slow or unstable, which might result in delays for the user. Users might also want to worry about uploading the scanned QR images to a remote server, particularly if the content is sensitive. In the paper, the authors state that there are trade-offs between convenience, privacy and inference delay within backend architectures. That doesn't mean the current architecture is ideal, but it has its limitations.

Finally, the test was provided with datasets prepared by the team, not a very large public benchmark, which would give greater confidence in the generalizability of the system. The results reported are positive, but testing on other external sets would be beneficial to validate the models under different conditions and attacker strategies, as well as different domains. This is particularly true of cybersecurity systems, which can

appear more successful than they actually are when trained on a restricted set of data.

In general, the constraints are mostly scale of the datasets, external dependence and deployment realism. These are frequent problems in applied machine learning and security research and clearly indicate directions for improving.

6.5 Future Work and Recommendations

There are a number of potential avenues for future development and enhancement of QRShield and making it more effective.

First, the datasets can be extended by increasing the size and variety of QR codes that can be scanned in the real world. More data would help generalize the models, and enable them to be more robust to variations in lighting, angle, blur, print quality and tampering style. QR codes are an application that is used in many contexts, so there should be examples from posters, receipts, restaurant menus, product packaging, payment screens, etc. in future datasets. The paper suggests that expanded and more diverse training sets are an important future research direction, as wider data would enable models to better deal with real-world variations.

Second, there is an opportunity to add more behavioral contextual elements to the URL branch, such as a deeper redirect chain analysis, domain reputation signals, and webpage content analysis. The features might provide the classifier with more information about the destination than the original URL structure. For instance, redirect behavior can be used to identify hidden risk patterns, and reputation sources, or content signals, can be used to identify newly created and evasive malicious domains. Based on the literature review, additional threat intelligence sources may enhance malicious URL detection further. These sources may increase the resilience of the system to the changing methods of attack.

Third, the deep learning side can be expanded by other lightweight or more sophisticated architectures, if the system is subsequently optimised for on-device or low resource deployment. The paper mentions that compression, pruning and quantization of models can aid in reducing the inference cost with minimal loss of accuracy. This is particularly applicable when QRShield is adapted for applications where faster processing or limited hardware resources are critical. A lighter version would make the mobile version more

efficient, and a more sophisticated architecture would further enhance the detection efficiency (if computational resources permit it).

Fourth, the fusion strategy can be further improved. Decision level fusion has achieved satisfactory practical value, but future versions may consider using more adaptive learned fusion methods, particularly in the absence of or when one modality is less reliable. This may help to boost robustness if the URL is not available or if the QR image is not easy to decode. A more flexible fusion layer could also enable the system to be better equipped to deal with the uncertainties in the predictions, as it would provide more context to the final decision layer. This can be especially useful in actual implementation if the image branch is able to identify tamper, but the URL branch is unable to return full information, or if the QR code is readable but is damaged visually.

Fifthly, more widely deployed real-world testing should be performed. The system is already a web application and a mobile application, so future work can be directed towards usability studies, latency testing and analyzing user feedback in order to gain insight into the system's performance in real-life situations. The paper also emphasizes that in practice accuracy is not the only issue to consider; throughput, latency and memory usage are equally significant. These tests would inform about its ability to respond, its ease of use for users and whether it maintains reliability after multiple real-life scanning sessions.

Lastly, the SHAP explainability module will be improved to better explain the prediction process to both the technical analysts and general users. The system would be even more transparent and trustworthy if there were more detailed visual explanations in the web app and less wordy explanations in the mobile app. This is crucial since explainability can boost user confidence in the system and enable users to grasp the rationale behind flagging a QR code. Future versions may display explanations in a more interactive manner, allowing for easier observations of the influence that certain features had on the final outcome.

In the overall future work directions, the efforts are centered on boosting data diversity, feature richness, deployment efficiency, fusion flexibility, realistic testing and explainability. These improvements would render QRShield more powerful and fit for application in the real-world on a large scale.

6.6 Concluding Remarks

QRShield proves that QR code security can be best achieved by a hybrid approach rather than a single-modality approach. The framework incorporates the image-based deep learning QR scheme with URL-based machine learning to mitigate against both malicious payload attacks and visual tampering attacks. This is significant because QR threats can manifest in a variety of ways, and a system focused on one element could overlook some of the threat. The proposed framework thus provides a more comprehensive and realistic defense approach.

The results of the experiments indicate that the models selected have achieved good performance, with XGBoost achieving better results for malicious URL detection and ResNet-18 better results for QR tampering detection. The obtained results confirmed the selection of the models in the final model and demonstrate the contribution of both branches: semantic and visual in the overall system. The final fusion layer makes the system more reliable and more applicable for real-world use as it fuses both of the sources of evidence before generating the final label.

SHAP-based explainability enhances the framework by clarifying the predictions. This is useful in a security environment since users will be more inclined to trust the system if it can explain its decisions. The proposed system is also deployable and user friendly, as both web and mobile applications are available. It is a research model as well as practice which can be applied in different environments.

In summary, the thesis has introduced a practical and modular QR security solution which can be used as a solid base for continued research and deployment in the real world in the field of cyber defense. It demonstrates the benefits of integrating machine learning, deep learning, fusion, and explainability to provide a more robust defense against contemporary QR-based attacks.

Bibliography

[1] Z. Mukhammedali, V. Khan, A. Razaque, M. A. Khan, and Z. Kalpeyeva, "Ai-enhanced security framework for qr codes," in *Proceedings of the IEEE SATC Conference*, February 2025, pp. 1–5.

[2] F. Trad and A. Chehab, "Detecting quishing attacks with machine learning techniques through qr code analysis," May 2025.

[3] F. Ahmed, I. Fatima, M. U. Hassan, I. Rasheed, and U. Fayyaz, "Malicious qr code detection: A comprehensive review of methods and challenges," dec 2025. [Online]. Available: https://doi.org/10.5281/zenodo.18196524

[4] A. K. Khan, "Detecting phishing urls in qr codes using heuristic techniques," Master's thesis, National College of Ireland, Dublin, Ireland, September 2023. [Online]. Available: https://norma.ncirl.ie/7127/

[5] H. Almousa, A. Almarzoqi, A. Alassaf, G. Alrasheed, and S. Alsuhibany, "Qr shield: A dual machine learning approach towards securing qr codes," *International Journal of Computing and Digital Systems*, vol. 15, pp. 887–898, August 2024.

[6] D. Lourenco, M. Sriraj, K. Thambi, and V. Ranjan, "Malicious urls and qr code classification using machine learning and deep learning techniques," in *IEEE Asian Conference (ASIANCON)*, August 2023, pp. 1–10.

[7] K. Kalpana, V. Maheshwaram, and K. Umarani, "Secure qr code scanner to detect malicious url using machine learning," *International Journal of Communication Networks and Information Security*, vol. 15, no. 4, pp. 647–662, December 2023. [Online]. Available: https://ijcnis.org/index.php/ijcnis/article/view/7652

[8] J. Ratna Kumari, S. L. Kambala, S. Paila, G. Kunkudu, and V. A. Chekuri, "Qr code malicious url detection system using machine learning," *International Journal of Communication Networks and Information Security*, vol. 17, no. 3, pp. 198–202, March 2025. [Online]. Available: https://ijcnis.org/index.php/ijcnis/article/view/8103

[9] B. Wang, "Malicious url detection with explainable machine learning techniques," in *Proceedings of the ACM Conference on Security and Data Mining*, July 2025, pp. 293–299.

[10] K. Berkani and R. Zeraoulia, "A hybrid deep learning and anomaly detection framework for real-time malicious url classification," November 2025.

[11] N. Nigam, "Performance analysis of qr phishing detection approaches," *Journal of Information Systems Engineering and Management*, vol. 10, pp. 221–225, April 2025.

[12] A. Trivedi, K. Jangal, and R. Gupta, "Phishing detection in advanced qr code attacks: Challenges and ai-driven solutions," *International Journal for Research in Applied Science and Engineering Technology*, vol. 13, January 2025.

[13] M. Sarkhi and S. Mishra, "Detection of qr code-based cyberattacks using a lightweight deep learning model," *Engineering, Technology & Applied Science Research*, vol. 14, pp. 15 209–15 216, August 2024.

[14] A. Alsulami, Q. Abu Al-Haija, B. Alturki, A. Yafoz, A. Alqahtani, R. Alsini, and S. Binaymin, "Efficient malicious qr code detection system using an advanced deep learning approach," *Computer Modeling in Engineering & Sciences*, vol. 145, pp. 1117–1140, October 2025.

[15] Y. Tashtoush, M. Khashim, and N. Alsaedi, "Comparing machine learning and deep learning models for qr code phishing detection," in *IEEE International Conference on Cyber-AI*, September 2025, pp. 291–297.

[16] O. Lamrabti, A. Mezrioui, and A. Belmekki, "Url_trigger: Real time solution for detection of malicious urls using deep learning," in *Advances in Intelligent Systems and Computing*, February 2024, pp. 328–334.